Hudson River Trading — WiTTI Onsite Interview

On campus, 60 minutes · part 1: a probability question · part 2: a programming exercise · bring your laptop · prepared Oct 25, 2026 · builds on the CodeSignal OA prep document

Part 1 — What this interview is

You passed the CodeSignal OA. The next step is a one-hour, in-person interview at your university with one HRT interviewer, in two parts: a probability question, then a programming exercise on your own laptop. This document collects every reported HRT probability and live-coding question found online, solves each one with a full explanation, and checks every answer by exact computation or simulation.

> **EVIDENCE** The only WiTTI-specific report found anywhere is a Wall Street Oasis thread whose poster quotes the WiTTI interview invitation: it "says Probability and a programming exercise" — the same two parts you were given. No one posted the actual questions. The one Glassdoor WiTTI review covers only the OA. So the questions below come from HRT interviews for neighbouring roles, weighted toward the closest analogues: one-hour intern, sophomore and early-career rounds that also paired one probability question with one coding task.

The closest analogues — one-hour HRT rounds with the same probability + coding shape

Date	Role (Glassdoor unless noted)	What was reported
Sep 2026	Data Scientist (PracHub report)	First Zoom round: estimate μ for a point-mass + normal mixture, **mean or median? then write a simulation** comparing their errors (P16).
Jul 2026	C++ SWE intern (PracHub report)	One-hour Zoom coding round: **root-to-leaf path sum in a binary tree**, then return all such paths (P17). "Felt like a LeetCode Easy"; edge cases mattered most.
Jul 2026	Sophomore intern	Statistics + simulation: **come up with an estimator, then code it to check its bias and variance**. Second round: questions about C.
May 2026	Explore HRT (first/second-year program)	"Write a simulation code, about stats." CoderPad; interviewer gave hints.
May 2026	Algo Developer intern	"Only one big question ... then the interviewer asks you to **implement a tiny simulation in Python** where you run some experiments on the question you were just asked."
Mar 2026	Winternship intern	CodeSignal OA, then **one hour: one probability question and one coding problem that was also kind of mathy**.
Jan 2026	Internship	Basic probability (**dice rolling — probability two dice match**) + a basic easy–medium coding question; partly behavioural.
Jan 2026	Algo Developer (Quantt)	First round: "basic stats question + coding part where you **run simulation to check your intuition**."
Nov 2025	Quantitative Researcher	Probability question, then **debug the simulation code** for it. Round 2: derive an **optimal dice-game strategy, then live-code to validate it**.
Apr 2025	Algo Developer intern	Live coding (45 min): **code an optimal strategy for a simplified version of blackjack**.
Feb 2024	Sophomore Program	OA, then a final interview with **one live coding problem**: a digit-divisibility counting question (see P11).

KEY Read together, these say the two halves are often linked: you derive a probability answer by hand, then write code that checks it (a Monte Carlo simulation) or computes it (a small dynamic program). The laptop instruction fits this. Prepare for both versions of part 2: (a) "simulate what you just derived" and (b) a short self-contained implementation problem — a game, a data structure, or a counting search.

For reference: the summer-internship final round is different

Quantt's HRT internship guide relays one detailed candidate report of the **summer Algorithm Developer intern final round**: four one-hour rounds, and the day may be cut short if the morning goes badly.

- **Programming:** an existing C++ client-and-server trading codebase supports adding and cancelling orders; implement "modify order".
- **Data analysis:** explore a dataset (a heart-disease prediction set) in a Jupyter notebook with pandas and seaborn; documentation allowed.
- **System design** and **maths**.

KEY That is the multi-round final for the summer program, not your one-hour WiTTI interview, so it is not the focus of this document. It is useful if WiTTI leads to a summer interview later, and it shows what HRT values: extending code someone else wrote, and reasoning about real data.

Laptop setup — do this the night before

- **Python 3 working offline**, with a plain editor you are fast in (VS Code is fine) and a terminal you can run python3 file.py in. Assume you may be asked to use CoderPad in the browser instead — check that one CoderPad pad opens on campus Wi-Fi.
- **numpy installed** (import numpy as np works). Do not rely on scipy — most simulations need only random and numpy.
- **Type out the Monte Carlo harness in P1 once from memory** so you can produce it in two minutes on the day. Do not bring pre-written solution files: HRT bans AI tools and expects your own work, and arriving with saved answers would look bad.
- Charger, notifications and chat apps off, large font in the editor so the interviewer can read your screen, and a few blank sheets for the probability part.
- If the interviewer asks for C++ instead, it is acceptable to say Python is your strongest interview language and ask whether that is fine — HRT lets most candidates choose.

The most important ones

Priority	Do this	Why
1	Type the **P1 Monte Carlo harness** from memory and run it on A2 (expected rolls to a sum of 7).	The single most reported pattern: "now simulate it."
2	Derive **A6 (two 6s in a row, E = 42)** by hand with states, then write its simulation and check it gives about 42.	The exact "derive, then simulate" format the Jan 2026 report describes — both halves of the interview in one problem.
3	Re-derive **C1 (taller person, E = ∞)**, **B1 (3 before 6, E = 9)** and **E1 (dice sum vs coin heads, Central Limit Theorem)**.	The most frequently reported HRT probability questions across years.
4	Run through **G1 / P2 (estimator bias and variance by simulation)** and **P16 (mean vs median, simulated)**.	The July 2026 sophomore and September 2026 reports — closest in format and date to WiTTI.
5	Read **P3 (dice game optimal strategy)** and the logic of **P4 (blackjack DP)**.	Strategy-then-validate is HRT's signature live-coding format.

Priority	Do this	Why
6	Write **P17 (tree path sum)** from scratch in 10 minutes, testing the edge cases out loud.	The July 2026 one-hour intern coding round.
7	Skim the trap boxes of A1–A7.	Warm-ups are where careless answers lose points.

Part 2 — The 60-minute playbook

The split below is a plan, not a reported schedule: interviewers vary, and the coding half may build on the probability half.

Minutes	Phase	What to do
0–5	Introductions	Thirty-second background: CS + Physics at Harvard, the Computational Robotics Lab work, Woman International Master. Then let them lead.
5–30	Probability question	Restate, clarify, solve a small case, give the answer with reasoning, sanity-check it, then handle follow-ups.
30–55	Programming exercise	Plan aloud for 2 minutes, write the simplest correct version, run it on a tiny case, then improve.
55–60	Your questions	Have three ready (below).

The probability half — five rules

1. **Restate the question in your own words and pin down every assumption.** Fair die? With or without replacement? "Expected number of rolls" including the last one? Interviewers often leave one detail ambiguous on purpose.
2. **Solve the smallest case first, out loud.** Two coins before ten; n = 2 before n. It gives you a number to check the general formula against.
3. **Name the tool you are using.** "I'll condition on the first roll", "by symmetry", "linearity of expectation over indicator variables", "set up states". HRT grades the method, not just the number.
4. **Sanity-check the answer before you announce it.** Is the probability between 0 and 1? Does it match the small case? Does it move the right way when a parameter changes?
5. **Offer the simulation yourself.** "If I had a laptop I'd check this with a quick Monte Carlo — want me to?" That bridges straight into part 2.

The programming half — five rules

1. **Ask for the input format and an example**, and write that example into the file as your first test.
2. **Say the plan and its complexity before typing.** One sentence: "I'll memoise on (total, upcard); that's 20 × 10 states."
3. **Get a correct brute force running first**, then improve it. A working slow version beats an unfinished clever one.
4. **Run it constantly.** You have a real laptop — use it. Print intermediate values; compare against your hand answer from part 1.
5. **Write readable code**: clear names, one function per idea, a docstring stating what it returns. HRT's own interview guide says it looks for idiomatic, well-encapsulated, readable code.

What HRT says it evaluates (from its own interview guide)

- **Collaboration:** do you take hints well and stay open to another approach?

- **Teachability:** do you apply an idea from earlier in the interview to the next step?
- **Communication:** can you explain your work at the right level of detail, and check that the interviewer is following?

> **SAY THIS** HRT explicitly asks candidates to over-communicate when they need quiet time to think, change approach, realise a mistake, do not know a term, or **have heard the problem before** — honesty is something they say they value.

Three questions to ask at the end

- "What do you think has made HRT so successful? And what do you enjoy most about working here?"
- "What does a WiTTI project look like in the first week — are interns mostly writing code, or working closer to signal research?"
- "How do HRT developers usually check a model or strategy before trusting it — is simulation like what we did today part of the daily work?"

Part 3 — Reported probability questions, solved

Thirty-three questions in seven groups, each reported by an HRT candidate (source and date under every title). Each one follows the same layout: the question stated precisely, the answer, the solution step by step, and the trap or follow-up the interviewer is likely to add. Every answer was checked by exact computation and by simulation before it was written here.

> **KEY** Groups A–C are the most often reported and the most likely for a WiTTI-level hour. Groups D–G are the follow-up and harder end. HRT probability questions are mostly variations on classics from Zhou's A Practical Guide to Quantitative Finance Interviews (the "green book") — several candidates say so directly.

A · Dice and coin warm-ups

The key idea. Count equally likely outcomes, or treat "repeat until success" as a geometric random variable: if each attempt succeeds with probability p, the expected number of attempts is 1/p.

A1. Probability that two dice match

Reported: *Glassdoor, Internship, Jan 2026 — "basic probability questions (dice rolling)"*

The question.

- Roll two fair six-sided dice once.
- What is the probability that both dice show the same number (a double)?

Answer. **1/6.**

Solution. Whatever the first die shows, the second die matches it with probability 1/6. Counting gives the same: 6 matching outcomes out of 36.

FOLLOW-UP QUESTIONS

▶ **"What about three dice — all three the same?"**
The first die can be anything; the second must match it (1/6) and so must the third (1/6): $1/6 \times 1/6 = \mathbf{1/36}$ (6 favourable outcomes out of 216).

▶ **"All three different?"**
The second die must avoid one value (5/6) and the third must avoid two (4/6): $5/6 \times 4/6 = \mathbf{5/9}$ (equivalently 6·5·4 = 120 out of 216).

▶ **"At least two the same?"**
This is the complement of "all different": $1 - 5/9 = \mathbf{4/9}$. Using the complement is the quick route — counting "exactly two" and "all three" separately also works ($90/216 + 6/216 = 96/216 = 4/9$) and is a good check.

A2. Expected number of rolls of two dice until they add up to 7

Reported: *DEV Community write-up of an HRT video interview, Nov 2025 (with the follow-ups below)*

The question.

- Throw a pair of fair six-sided dice together, again and again; one "roll" means one throw of both dice.
- Stop as soon as the two dice add up to 7.
- What is the expected number of rolls, counting the final roll that gives 7?

Answer. **6.**

Solution. Six of the 36 outcomes sum to 7 — (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) — so each roll succeeds with $p = 1/6$. The number of rolls is geometric, with mean $1/p = 6$.

FOLLOW-UP QUESTIONS

▶ **"How long until a 7 on two rolls in a row?"**
Define two states, based only on what the most recent roll was — that is all the future depends on:

$\mathbf{E_0}$ = the expected number of **additional** rolls needed to finish, starting from a position where the last roll was **not** a 7. This includes the very start of the game, before any roll (there is no streak yet).

$\mathbf{E_1}$ = the expected number of additional rolls needed when the last roll **was** a 7 — you are halfway there and need one more 7.

The answer to the question is E_0, because the game starts in state 0.

Condition on the next roll. Whatever happens, that roll counts as 1. Then add the expected rolls still needed from the state you land in.

From state 1 (last roll was a 7): with probability 1/6 the next roll is a 7 — that roll finishes the game, so the total from here is just 1. With probability 5/6 it is not a 7 — the streak is broken, you have used 1 roll and are back in state 0, needing E_0 more:

$$E_1 = \frac{1}{6} \cdot 1 + \frac{5}{6}(1 + E_0) = 1 + \frac{5}{6} E_0$$

From state 0 (no streak): with probability 1/6 the next roll is a 7 — you have used 1 roll and moved to state 1, needing E_1 more. With probability 5/6 it is not — you have used 1 roll and are still in state 0:

$$E_0 = \frac{1}{6}(1 + E_1) + \frac{5}{6}(1 + E_0) = 1 + \frac{1}{6} E_1 + \frac{5}{6} E_0$$

The two ways of writing each equation are the same: the probabilities add to 1, so the "1 roll used" term can be pulled out in front. The first form makes the logic of each branch explicit; the second is quicker to solve.

Solve. The E_0 equation simplifies to $E_0 = 6 + E_1$. Substitute into $E_1 = 1 + (5/6)E_0$: $E_1 = 36$, so $\mathbf{E_0 = 42}$.

▶ **"What if the dice are biased?" (back to the main question: rolls until one 7)**

Only the per-roll success probability changes: $p = \Sigma_i$ P(die 1 = i) · P(die 2 = 7 − i), and the answer is 1/p. A neat observation: **if either die is fair**, p is still 1/6 — each face of the other die is completed by exactly one face of the fair die, which comes up with probability 1/6 whatever the other die does.

For **two 7s in a row** (the follow-up above), plug the same p into $(1 + p)/p^2$. If either die is fair, p is still 1/6, so the answers stay 6 and 42.

▶ **"What if you may keep one die and re-roll only the other?" (also back to the main question: rolls until one 7)**

Whatever value the kept die shows, exactly one face of the re-rolled die makes 7, so each re-roll still succeeds with probability 1/6 and the expected number of rolls is still **6**. Keeping a die does not help — the point the interviewer wants you to see quickly.

A3. Expected rolls to see a 1, and to see every face

Reported: *Glassdoor, Quantitative Researcher, Jan 2022; Algorithm Development Internship, Dec 2023*

The question.

- (a) Roll a fair six-sided die until it shows a 1. What is the expected number of rolls, counting the roll that shows the 1?
- (b) Roll a fair six-sided die until each of the faces 1, 2, 3, 4, 5, 6 has appeared at least once. What is the expected total number of rolls?

Answer. (a) 6. (b) 14.7.

Solution. (a) Geometric with p = 1/6. (b) Split the wait into stages. After k distinct faces have appeared, a new face appears with probability (6 − k)/6, so that stage takes 6/(6 − k) rolls on average. Summing over k = 0…5:

$$E = \frac{6}{6} + \frac{6}{5} + \frac{6}{4} + \frac{6}{3} + \frac{6}{2} + \frac{6}{1} = 6\left(1 + \frac{1}{2} + \cdots + \frac{1}{6}\right) = 6 \times 2.45 = 14.7$$

> **TRAP** The stages are not equally long — the last new face alone takes 6 rolls on average. This is the coupon-collector problem: for n faces the answer is $n \cdot H_n \approx n \ln n$.

A4. Probability that the sum of two dice is greater than their product

Reported: *LinkJob candidate write-up, 2025 HRT quant interview*

The question.

- Roll two fair six-sided dice; call the numbers they show a and b.
- What is the probability that their sum is greater than their product, i.e. $a + b > a \times b$?

Answer. **11/36.**

Solution. $a + b > ab$ is the same as $(a-1)(b-1) < 1$, i.e. $(a-1)(b-1) = 0$, i.e. at least one die is 1. P(at least one 1) $= 1 - (5/6)^2 = 11/36$.

> **TRAP** Check (2, 2): 4 is not greater than 4. The rearrangement to $(a-1)(b-1) < 1$ is the fast route — it avoids listing cases.

A5. Expected sum of all the faces rolled before the first 1

Reported: *QuantVault HRT list, "Expected Sum Until Rolling a 1", last seen 2024*

The question.

- Roll a fair six-sided die again and again, and stop as soon as it shows a 1.
- Add up the faces of all the rolls **before** that 1 (the final 1 is not included). Example: rolls 4, 6, 2, 1 give a sum of $4 + 6 + 2 = 12$; if the first roll is a 1, the sum is 0.
- What is the expected value of this sum?

Answer. **20** (21 if the final 1 is counted).

Solution. Let N be the number of rolls up to and including the first 1. Each roll is a 1 with probability 1/6, independently, so N is geometric with $\mathbf{E[N] = 1/(1/6) = 6}$. Of those rolls, the last one is the 1 and the other $N - 1$ are the rolls being added up, so on average there are $\mathbf{6 - 1 = 5}$ of them.

Method 1 (Wald's identity). First find the expected total **including** the final 1. The rolls are independent, and the decision to stop depends only on rolls already seen, so E[total including the 1] = E[N] × E[one roll] = 6 × 3.5 = 21. The final roll is always exactly 1, so remove it: $\mathbf{21 - 1 = 20}$.

Method 2 (check: count the added rolls directly). On average there are 5 rolls before the 1, and each of them is known **not** to be a 1, so it is equally likely to be 2, 3, 4, 5 or 6, with average $(2 + 3 + 4 + 5 + 6)/5 = 4$. So E[sum] = 5 × 4 = **20**. Adding the final 1 gives 21 again, matching Method 1.

Method 3 (one equation, by conditioning on the first roll). Let E be the answer. The first roll is a 1 with probability 1/6, and then the sum is 0. Otherwise (probability 5/6) it is a 2–6, averaging 4, and the game starts afresh, so the remaining expected sum is again E:

$$E = \frac{1}{6} \cdot 0 + \frac{5}{6}(4 + E) \quad \Longrightarrow \quad \frac{E}{6} = \frac{10}{3} \quad \Longrightarrow \quad E = 20$$

> **KEY** **Including the final roll, the expected sum is 21 whichever number you stop on** — a 1 here, a 6 in your Weiss prep (P7), or any other face. Wald's identity gives E[N] × 3.5 = 6 × 3.5 = 21 in every case, because the stopping face only decides when you stop, not how many rolls you expect. The answer depends on the face only once the final roll is removed: 21 − k when you stop on k, so **20** for a 1 and **15** for a 6 (5 × 3, since rolls before a 6 average 3).

> **TRAP** The rolls being added are known not to be 1, so each averages 4, not 3.5. Using 3.5 gives 5 × 3.5 = 17.5 — the most common wrong answer. Equally, do not use 6 × 4 = 24: that counts the final roll as if it were an ordinary non-1 roll.

A6. Expected rolls until two sixes in a row — derive it, then simulate it

Reported: *Quantt HRT internship guide (Oct 2026): written by Quantt as an example of the reported January 2026 round, "basic stats question + coding part where you run simulation to check your intuition" — the format is reported, this exact question is Quantt's illustration*

The question.

- Roll a fair six-sided die again and again.
- Stop as soon as two consecutive rolls are both 6. Example: 6, 3, 6, 6 stops after 4 rolls.
- What is the expected number of rolls, counting every roll including both 6s?
- Then: write a quick simulation to check your answer.

Answer. 42.

Solution. Use two states. E_0 = expected rolls still needed when the last roll was not a 6 (including the start); E_1 = expected rolls still needed when the last roll was a 6.

$$E_1 = 1 + \frac{1}{6}\cdot 0 + \frac{5}{6}E_0 \qquad \text{(a 6 finishes; anything else sends you back to the start)}$$

$$E_0 = 1 + \frac{1}{6}E_1 + \frac{5}{6}E_0 \qquad \text{(a 6 moves you to state 1; anything else keeps you at the start)}$$

From the second equation, $(1/6)E_0 = 1 + (1/6)E_1$, so $E_0 = 6 + E_1$. Substitute into the first: $E_1 = 1 + (5/6)(6 + E_1)$, so $(1/6)E_1 = 6$ and $E_1 = 36$. Then $E_0 = 42$. In general, with success probability p per roll, $E_0 = (1 + p)/p^2 = (1 + 1/6)\cdot 36 = 42$.

The simulation. Keep a flag for "the previous roll was a 6"; stop when the current roll is a 6 and the flag is set. Average over many games and report a 95% interval.

```python
import random

def rolls_until_double_six():
    rolls, last_six = 0, False
    while True:
        rolls += 1
        six = random.randint(1, 6) == 6
        if six and last_six:
            return rolls
        last_six = six
```

```python
if __name__ == "__main__":
    random.seed(0)
    trials = 200_000
    xs = [rolls_until_double_six() for _ in range(trials)]
    mean = sum(xs) / trials
    sd = (sum((x - mean) ** 2 for x in xs) / trials) ** 0.5
    print(f"{mean:.2f} +/- {1.96 * sd / trials ** 0.5:.2f}   (exact 42)")
# output:  41.98 +/- 0.18   (exact 42)
```

> **TRAP** The common wrong answer is 36, from "two 6s have probability 1/36, so wait 36 rolls". That would be right if you rolled pairs of dice in separate blocks, but here a failed attempt can overlap the next one, and a non-6 after a 6 wastes the progress. Saying why 36 is wrong is part of a good answer.

FOLLOW-UP QUESTIONS

▶ **"What about a 6 followed by a 5 (rather than two 6s)?"**
Same two-state method. E_0 = expected rolls from scratch, E_1 = expected rolls when the last roll was a 6. From E_0: $E_0 = 1 + (1/6)E_1 + (5/6)E_0$, so $E_0 = 6 + E_1$. From E_1: a 5 finishes $(1/6)$, **another 6 keeps you in state E_1** $(1/6)$, anything else sends you back to E_0 $(4/6)$: $E_1 = 1 + (1/6)E_1 + (4/6)E_0$. Substituting $E_0 = 6 + E_1$ gives $E_1/6 = 5$, so $E_1 = 30$ and $\mathbf{E_0 = 36}$.

▶ **"Why is it shorter than two 6s (42)?"**
With "66", a failure after a 6 throws away your progress. With "65", the most common near-miss (another 6) keeps your progress. Patterns that cannot overlap with themselves are found sooner — the same reason HT (4 flips) beats HH (6 flips) in C2.

A7. Probability that the total of 10 dice rolls is a multiple of 7

Reported: *Reddit r/quantfinance and r/learnquant posts "HRT Quant Interview Question", images from the prep site quantprof.org — a prep site labels it HRT; not a first-hand report*

The question.

- Roll a fair six-sided die 10 times and add up the 10 numbers. The total is somewhere between 10 and 60.
- What is the probability that this total is divisible by 7, i.e. that it equals 14, 21, 28, 35, 42, 49 or 56?

Answer. $\mathbf{1/7 + (6/7)\cdot(1/6)^{10} = 1{,}439{,}671/10{,}077{,}696 \approx 0.142857157}$ — just above 1/7.

Solution. Track only the remainder of the running total mod 7. Let $p_n = P(\text{total after n rolls} \equiv 0 \bmod 7)$, with $p_0 = 1$.

1. If the total so far is **not** a multiple of 7, exactly one of the six faces makes the new total a multiple of 7 (the faces 1–6 hit every non-zero remainder exactly once). Probability 1/6.
2. If the total so far **is** a multiple of 7, no face can keep it one (that would need a 0 or a 7). Probability 0.

$$p_n = \frac{1 - p_{n-1}}{6}, \qquad p_0 = 1$$

This recursion pulls p_n toward its fixed point $p = (1 - p)/6$, i.e. $p = 1/7$, and the gap shrinks by a factor $-1/6$ each roll: $p_n = 1/7 + (6/7)(-1/6)^n$. For n = 10 the correction is $(6/7)/6^{10} \approx 1.4 \times 10^{-8}$. An exact count over all 6^{10} outcomes (dynamic programming on the remainder) gives the same fraction.

> **SAY THIS** "The answer is essentially 1/7, and here is exactly how far off it is." Interviewers want both the intuition (remainders mix almost perfectly) and the exact recursion. Check small cases: $p_1 = 0$ and $p_2 = 1/6$ (six pairs out of 36 sum to 7).

B · Conditioning and Bayes

The key idea. Write $P(A \mid B) = P(A \text{ and } B)/P(B)$, or with Bayes' rule, posterior $\propto$ prior $\times$ likelihood. List the hypotheses, weight each by how likely the evidence is under it, then normalise.

B1. Expected rolls to the first 6, given that a 3 appeared before it

Reported: *Glassdoor, Quantitative Trader phone interview, Aug 2026 (the most recent reported HRT probability question)*

The question.

- Roll a fair six-sided die again and again, and stop when the first 6 appears.
- You are told that at least one 3 was rolled before that first 6.
- Given this information, what is the expected number of rolls, counting the final 6?

Answer. **9** (the unconditional answer would be 6).

Solution. Only the faces 3 and 6 matter for the condition. Split the rolls into two stretches.

1. **Up to the first roll that is a 3 or a 6.** The question says a 3 appears before the first 6. Ignore every roll that is neither 3 nor 6 (1, 2, 4, 5 change nothing). Then the condition is exactly: **among the rolls that show a 3 or a 6, the first one is a 3.** (Not that the very first roll of the game is a 3 — any number of 1s, 2s, 4s and 5s may come before it.) Each roll is a 3 or a 6 with probability 2/6 = 1/3, so this stretch takes 3 rolls on average. When that roll arrives, it is a 3 or a 6 with equal chance, 1/2 each, and which one it is has nothing to do with how many rolls the stretch took. So conditioning on "it was a 3" does not change the stretch's expected length: still **3**.
2. **From that 3 to the first 6.** Now the condition is satisfied and the die starts fresh: a 6 arrives after 6 rolls on average.

$$E[\text{rolls} \mid \text{a 3 came before the first 6}] = 3 + 6 = 9$$

TRAP The tempting answer is "more than 6, but I can't say how much." The decomposition works because, for a geometric race between two outcomes, **which** outcome happens first is independent of **when** it happens. Simulating 400,000 games gives 8.999.

B2. The painted cube: five white faces visible

Reported: *Glassdoor, Algorithm Developer phone interview, Jan 2022 (also reported as a 3 × 3 × 3 cube question the same month)*

The question.

- A 3 × 3 × 3 cube is painted red on its outside, then cut into 27 small unit cubes. A small cube has a red face only where that face was on the big cube's surface; all other faces are unpainted (white).
- One small cube is picked at random and tossed onto a table.
- You can see five of its faces — the top and the four sides — and all five are white.
- What is the probability that the hidden face, the one on the table, is red?

Answer. **1/2.**

Solution. Classify the 27 cubes by painted faces: 8 corners (3 red faces), 12 edges (2), 6 face-centres (1), 1 core (0). Seeing five white faces is possible only for the core or a face-centre:

- Core: prior 1/27, chance of five white faces = 1.
- Face-centre: prior 6/27, chance = 1/6 (its single red face must be the one on the table).

$$P(\text{red underneath} \mid \text{five white}) = \frac{\frac{6}{27}\cdot\frac{1}{6}}{\frac{6}{27}\cdot\frac{1}{6}+\frac{1}{27}\cdot 1} = \frac{1/27}{2/27} = \frac{1}{2}$$

FOLLOW-UP QUESTIONS

▶ **"Same cube, no information: probability the top face is red?"**
Count unit faces instead of cubes: the 27 unit cubes have 27 × 6 = 162 faces, and the painted outside is 6 big faces × 9 = 54 unit faces. Each of the 162 faces is equally likely to end up on top, so P = 54/162 = **1/3**.

▶ **"Given the top face is red, what is the probability the cube is a corner?"**
Bayes with the painted-face counts as weights: corners contribute 8 × 3 = 24 red faces, edges 12 × 2 = 24, face-centres 6 × 1 = 6 (total 54). So P(corner | red on top) = 24/54 = **4/9**, edge 4/9, face-centre 1/9.

B3. Three bowls of balls

Reported: *Glassdoor, Quantitative Researcher, Apr 2021*

The question.

- Bowl 1 holds two blue balls, bowl 2 holds two red balls, and bowl 3 holds one red and one blue ball.
- Pick one of the three bowls at random, then draw one of its two balls at random: it is blue.
- What is the probability that the other ball in the same bowl is also blue?

Answer. **2/3.**

Solution. There are three blue balls in total, each equally likely to be the one drawn. Two of them sit in bowl 1, whose other ball is blue; one sits in bowl 3, whose other ball is red. So the answer is 2/3.

TRAP The intuitive answer 1/2 ("it is bowl 1 or bowl 3") ignores that bowl 1 is twice as likely to produce a blue draw. Same structure as the three-cards problem.

B4. A double-headed coin among fair coins

Reported: *Glassdoor, Algorithm Developer, Oct 2015; LinkJob 2025 write-up (second version)*

The question.

- (a) A jar holds 1,000 coins: one is double-headed (heads on both sides) and 999 are fair. You pick one coin at random and flip it 10 times; it lands heads all 10 times. What is the probability you picked the double-headed coin?
- (b) A jar holds 100 coins: 98 fair, one double-headed and one double-tailed (tails on both sides). You pick one coin at random and flip it once; it lands heads. What is the probability you picked the double-headed coin?

Answer. **(a) 1024/2023 ≈ 0.506. (b) 1/50.**

Solution. (a) Prior odds double-headed : fair = 1 : 999. Likelihood of 10 heads: 1 versus $(1/2)^{10} = 1/1024$. Posterior odds = 1 : 999/1024 = 1024 : 999, so P = 1024/2023.

(b) P(heads) = (98 × 1/2 + 1)/100 = 50/100. P(double-headed and heads) = 1/100. So P = (1/100)/(50/100) = 1/50.

FOLLOW-UP QUESTIONS

▶ **"How many heads in a row before you are 90% sure it is the double-headed coin?"**
After n heads the posterior is $P = 2^n/(2^n + 999)$ (prior odds 1 : 999, likelihood ratio 2^n). Requiring $P \geq 0.9$ means $2^n \geq 9 \times 999 = 8{,}991$. Since $2^{13} = 8{,}192$ is too small and $2^{14} = 16{,}384$ is enough, **n = 14**.

▶ **"And if a single tail appears?"**
The double-headed coin can never show tails, so the posterior drops to exactly **0** — one observation can rule out a hypothesis completely.

C · Expected value and waiting times

The key idea. Two tools solve almost all of these. **Linearity of expectation:** write the count as a sum of 0/1 indicators and add their probabilities — no independence needed. **Tail sums:** for a count $N \geq 0$, $E[N] = \Sigma_n P(N > n)$.

C1. How many people until someone taller? (the most reported HRT question)

Reported: *Glassdoor: Algo Developer Intern Oct 2022; Entry Level Quant Dev Sep 2022; Algorithm Developer Aug 2023 and Dec 2021; Algo Dev Oct 2021; Trading Algo Developer Jan 2021 (twice). QuantVault: "Expected People Passed Before Seeing Someone Taller", seen 2025*

The question.

- Pick a person at random from the street and note their height, X.
- Then keep picking more people at random, one at a time, and stop at the first person who is taller than X.
- Let N be the number of additional people you pick, counting that taller person (so N = 1 if the very next person is taller). What is E[N]?

Answer. **E[N] is infinite**, even though N is finite with probability 1 and is usually small.

Solution. $N > n$ means the first n further people are all shorter than X, i.e. X is the tallest of the first n + 1 people. By symmetry each of those n + 1 is equally likely to be the tallest, so $P(N > n) = 1/(n + 1)$. Then:

$$E[N] = \sum_{n \geq 0} P(N > n) = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots = \infty \qquad \text{(the harmonic series diverges)}$$

> **WHY** The intuition interviewers want: half the time X is below the median and you wait about 1 person, but sometimes X is the tallest person for miles and you wait forever. Those rare huge waits make the mean blow up. $P(N \geq 10)$ is exactly 1/10, and a simulation's running average never settles (1,000 trials gave 7.0; 200,000 gave 13.6).

FOLLOW-UP QUESTIONS

▶ **"If the mean is infinite, what is a sensible summary?"**
The **median**. The very next person is taller than X with probability 1/2 (by symmetry), so $P(N = 1) = 1/2$ and the median of N is 1. More generally $P(N > n) = 1/(n + 1)$: a 10% chance of waiting 10 or more people, 1% for 100 or more — heavy-tailed, which is why the mean diverges.

▶ **"What if you compare to your own height instead?"**
If your height is just another random draw, nothing changes: $E[N] = \infty$. But if your height h is a **known, fixed** number, each person is taller with a fixed probability $q = P(\text{height} > h)$, so N is geometric with mean **1/q** — finite. The infinity comes from averaging over the possibility that X itself is extremely tall.

C2. Waiting for coin patterns, and which pattern comes first

Reported: *Glassdoor, Algorithm Developer, Nov 2024 ("possibility of getting a certain pattern for flipping coin repeatedly"); QuantVault, "Waiting Time to Patterns HTH and HHH", seen 2023*

The question.

- Flip a fair coin repeatedly and watch the sequence of results.
- (a) What is the expected number of flips until the pattern HH (two heads in a row) first appears? Until HTH appears? Until HHH appears?
- (b) Keep flipping until either HTH or HHH appears as consecutive flips. What is the probability that HTH appears first?

Answer. (a) HH: 6, HTH: 10, HHH: 14 (HHT: 8). (b) 3/5.

Solution (a), by states for HH. Let E_0 = expected flips from scratch and E_1 = after one H. $E_1 = 1 + \frac{1}{2}\cdot 0 + \frac{1}{2}\cdot E_0$ and $E_0 = 1 + \frac{1}{2}\cdot E_1 + \frac{1}{2}\cdot E_0$. Substituting gives $E_0 = 6$.

The same method for longer patterns. Each state records how much of the pattern the most recent flips already match; E with that subscript is the expected number of **additional** flips needed from there. Each equation conditions on the next flip: it costs 1 flip, and each outcome (probability $\frac{1}{2}$) moves you to a new state. The key question for every state is where a **wrong** flip sends you — not always back to the start.

HTH. States: E_0 = nothing matched; E_1 = last flip H; E_2 = last two flips HT.

$$E_0 = 1 + \tfrac{1}{2}E_1 + \tfrac{1}{2}E_0 \qquad \text{(H starts the pattern; T leaves you at the start)}$$
$$E_1 = 1 + \tfrac{1}{2}E_2 + \tfrac{1}{2}E_1 \qquad \text{(T gives HT; another H still ends in H: stay in state 1)}$$
$$E_2 = 1 + \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}E_0 \qquad \text{(H completes HTH; T gives ...HTT, which matches nothing)}$$

Solve: $E_0 = 2 + E_1$, $E_1 = 2 + E_2$ and $E_2 = 1 + \frac{1}{2}E_0$, so $E_0 = 2 + 2 + 1 + \frac{1}{2}E_0$, giving $\mathbf{E_0 = 10}$.

HHT. States: E_0 = nothing; E_1 = last flip H; E_2 = last two flips HH.

$$E_0 = 1 + \tfrac{1}{2}E_1 + \tfrac{1}{2}E_0$$
$$E_1 = 1 + \tfrac{1}{2}E_2 + \tfrac{1}{2}E_0 \qquad \text{(H gives HH; T gives ...HT, which matches nothing)}$$
$$E_2 = 1 + \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}E_2 \qquad \text{(T completes HHT; another H still leaves HH)}$$

Solve: $E_2 = 2$, so $E_1 = 1 + 1 + \frac{1}{2}E_0 = 2 + \frac{1}{2}E_0$, and $E_0 = 2 + E_1 = 4 + \frac{1}{2}E_0$, giving $\mathbf{E_0 = 8}$. It is fast because once you reach HH you can never lose it — you just wait for a T.

HHH. States: E_0 = nothing; E_1 = last flip H; E_2 = last two flips HH.

$$E_0 = 1 + \tfrac{1}{2}E_1 + \tfrac{1}{2}E_0$$
$$E_1 = 1 + \tfrac{1}{2}E_2 + \tfrac{1}{2}E_0 \qquad \text{(H gives HH; T resets)}$$
$$E_2 = 1 + \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}E_0 \qquad \text{(H completes HHH; T resets all the way to the start)}$$

Solve: $E_2 = 1 + \frac{1}{2}E_0$, so $E_1 = 1 + \frac{1}{2}(1 + \frac{1}{2}E_0) + \frac{1}{2}E_0 = 3/2 + \frac{3}{4}E_0$, and $E_0 = 2 + E_1 = 7/2 + \frac{3}{4}E_0$, giving $\mathbf{E_0 = 14}$.

Compare HHT (8) with HHH (14): both need HH first, but from HH a miss costs HHH everything, while for HHT the "miss" (another H) costs nothing.

Shortcut for any pattern. Add 2^k for every k at which the first k letters of the pattern equal its last k letters. HTH: $k = 1$ (H) and $k = 3 \to 2 + 8 = 10$. HHH: $k = 1, 2, 3 \to 2 + 4 + 8 = 14$. HHT: only $k = 3 \to 8$. Patterns that overlap themselves take longer.

Solution (b). Both patterns start with H, so wait for the first H. From state "H": next T → state "HT" (then H wins for HTH, T resets to start); next H → "HH" (then H wins for HHH, T goes to "HT"). Let a, b, c be P(HTH wins) from H, HT, HH: $a = \frac{1}{2}b + \frac{1}{2}c$, $b = \frac{1}{2}\cdot 1 + \frac{1}{2}a$, $c = \frac{1}{2}\cdot 0 + \frac{1}{2}b$. Solving gives $a = 3/5$.

> **TRAP** Waiting time and "which comes first" are different questions. HH takes 6 flips on average and TH only 4, but the race is more lopsided than those numbers suggest: **TH comes before HH with probability 3/4**. Unless the first two flips are HH, a T appears before any HH — and from then on the first HH must be preceded by TH.

C3. Cards turned until the first ace

Reported: *QuantVault, "Turning Cards Until the First Ace", seen 2022; LinkJob 2025 write-up*

The question.

- Shuffle a standard 52-card deck (4 aces) and turn the cards face up one at a time.
- What is the expected number of cards turned, counting the first ace itself?

Answer. **53/5 = 10.6.**

Solution. The 4 aces split the 48 other cards into 5 gaps (before the first ace, between aces, after the last). By symmetry each gap holds $48/5 = 9.6$ non-aces on average. The first ace comes after the first gap: $9.6 + 1 = 10.6$.

> **TRAP** Why not the geometric answer? A geometric distribution needs the **same** success probability on every try, which happens only when you draw **with replacement** (put each card back and reshuffle). Then $p = 4/52 = 1/13$ on every draw and the expected wait is $1/p =$ **13** cards. Here the cards are **not** put back: every non-ace you turn removes a non-ace, so the chance that the next card is an ace keeps rising — 4/52, then 4/51, 4/50, ... and after all 48 non-aces it is certain. The ace therefore tends to come sooner, and the answer 10.6 is below 13. (Also note 1/13 is the probability per draw, not the expected number of draws.)

FOLLOW-UP QUESTIONS

▶ **"General formula: n cards, k special ones?"**
The k special cards split the other $n - k$ cards into $k + 1$ gaps; by symmetry each gap holds $(n - k)/(k + 1)$ cards on average. The first special card comes after the first gap, so its expected position is $(n - k)/(k + 1) + 1 =$ **$(n + 1)/(k + 1)$**. Check: $n = 52$, $k = 4$ gives 53/5.

▶ **"Expected position of the last ace?"**
By the same gap argument (or symmetry — reverse the deck), it is $n + 1 - (n + 1)/(k + 1) = k(n + 1)/(k + 1) = 4 \times 53/5 =$ **42.4**.

C4. Uniform numbers until the sum passes 1

Reported: *QuantVault, "Expected Number of Uniforms to Exceed One", seen 2022*

The question.

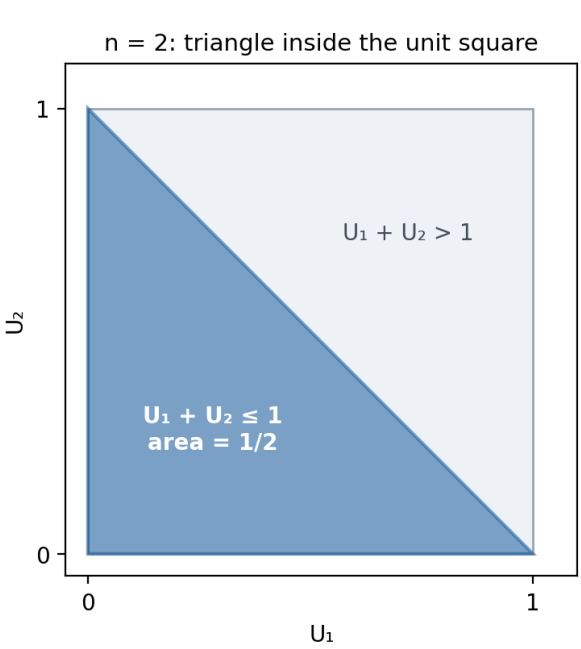
- Draw random numbers, each independent and uniform on $[0, 1]$, and keep a running sum.
- Stop as soon as the running sum is greater than 1.
- What is the expected number of numbers drawn, counting the one that pushes the sum past 1?

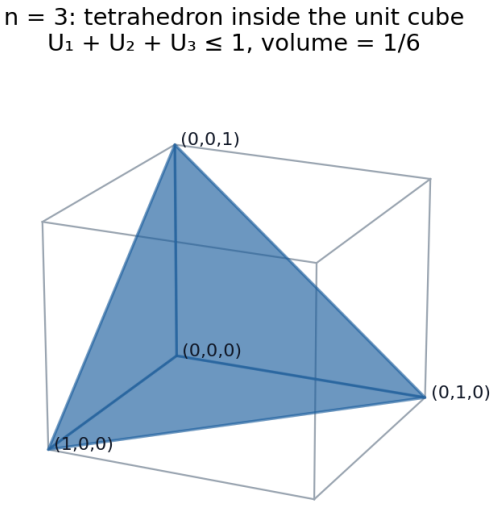
Answer. **$e \approx 2.718$.**

Solution. Use the tail-sum formula $E[N] = \Sigma_{n \geq 0} P(N > n)$, so we need $P(N > n)$ for each n.

1. **Translate the event.** $N > n$ means you have **not** stopped after n draws, i.e. the first n numbers add up to at most 1: $P(N > n) = P(U_1 + U_2 + \ldots + U_n \leq 1)$.
2. **Turn probability into volume.** The point $(U_1, \ldots, U_n)$ is uniformly spread over the unit cube $[0, 1]^n$, which has volume 1. So the probability of any region is just its volume. We need the volume of the region where every coordinate is ≥ 0 and their sum is ≤ 1.
3. **That region is a simplex** — the simplest solid with flat sides in n dimensions, cut off from the corner of the cube by the slanted plane "sum = 1". Its corners are the origin and the n points one unit along each axis. In 1-D it is the segment $[0, 1]$ (length 1); in 2-D the triangle with corners $(0, 0)$, $(1, 0)$, $(0, 1)$ (area 1/2); in 3-D the tetrahedron with corners $(0, 0, 0)$, $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$ (volume 1/6). See the figure.
4. **Its volume is 1/n!.** Slice the n-dimensional simplex at height $U_n = t$: the slice is an $(n - 1)$-dimensional simplex shrunk by the factor $(1 - t)$, so its volume is $V_{n-1}(1 - t)^{n-1}$. Adding up the slices, $V_n = \int_0^1 V_{n-1}(1 - t)^{n-1}\, dt = V_{n-1}/n$. Starting from $V_1 = 1$: $V_2 = 1/2$, $V_3 = 1/6$, ..., $V_n = 1/n!$. (Symmetry check: the cube splits into n! equally likely orderings of the coordinates, and the simplex has the same volume as one of them.)
5. **Add the tails.** $P(N > 0) = 1$ and $P(N > 1) = 1$ (a single number never exceeds 1), $P(N > 2) = 1/2$, $P(N > 3) = 1/6$, ..., so

$$E[N] = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \cdots = e \approx 2.718$$





$P(N > n)$ is the volume of the shaded simplex: 1/2 when $n = 2$ (triangle), 1/6 when $n = 3$ (tetrahedron).

> **KEY** Two quick consequences worth quoting: you always need at least 2 draws, and $P(N = 2) = P(N > 1) - P(N > 2) = 1 - 1/2 = 1/2$. A 200,000-run simulation gives 2.720.

C5. Expected number of inversions in a random permutation

Reported: *Glassdoor, Quant Researcher intern, Aug 2025*

The question.

- Shuffle the numbers 1, 2, ..., n into a uniformly random order.
- An inversion is a pair of positions $i < j$ where the larger number comes first. Example: the order 3, 1, 2 has two inversions, $(3, 1)$ and $(3, 2)$.
- What is the expected number of inversions?

Answer. $\mathbf{n(n-1)/4}$.

Solution. Use one indicator per pair of positions: there are $C(n, 2)$ pairs, and each pair is out of order with probability $1/2$ by symmetry. Linearity gives $C(n, 2)/2 = n(n-1)/4$. (For $n = 6$ the brute-force average over all 720 permutations is exactly 7.5.)

FOLLOW-UP QUESTIONS

▶ **"What is the variance of the number of inversions?"**
$n(n-1)(2n+5)/72$. The pair indicators are not independent, so a different decomposition is needed: for each position j, let C_j = the number of earlier values larger than the j-th value. For a random permutation, C_j is uniform on $\{0, 1, \ldots, j-1\}$ and the C_j are **independent** (this is the "inversion table" of the permutation). A uniform on j values has variance $(j^2-1)/12$, so $\mathrm{Var} = \Sigma_{j=1}^{n} (j^2-1)/12$, which simplifies to $n(n-1)(2n+5)/72$. For $n = 6$ that is $85/12 \approx 7.08$.

▶ **"How would you count inversions in code?"**
A double loop is $O(n^2)$; merge sort counts them in $O(n \log n)$ — see P25 in Part 4.

C6. Random chords in a circle: expected intersections

Reported: *Glassdoor, Python Algorithm Developer intern phone screen, Feb 2024; QuantVault, "Expected Number of Chord Intersections Inside a Circle", seen Jun 2026; Reddit r/quantfinance and r/learnquant posts "HRT Quant Interview Question", images from the prep site quantprof.org (the 10-chord version)*

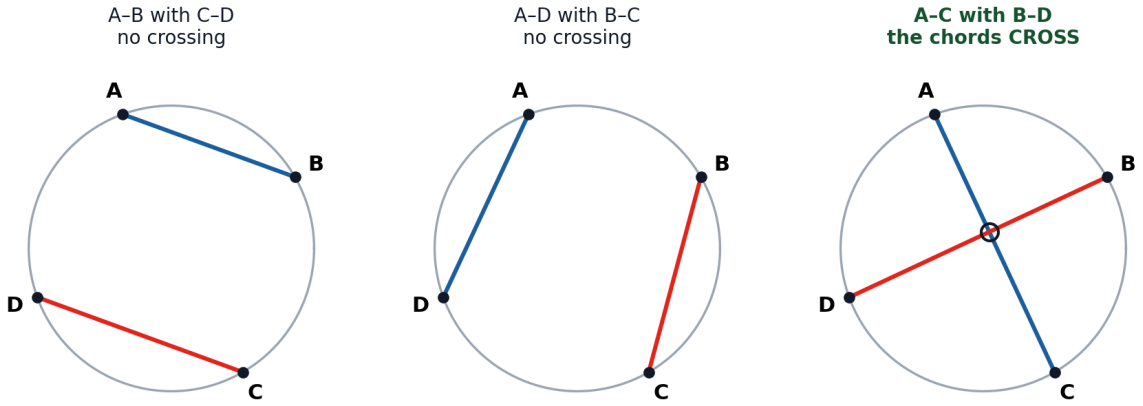
The question.

- Draw n chords in a circle. Each chord joins two points chosen independently and uniformly at random on the circumference.
- What is the expected number of points inside the circle where two chords cross?

Answer. $\mathbf{C(n, 2)/3 = n(n-1)/6}$.

Solution. Take any two chords: their four endpoints are four random points on the circle. There are three equally likely ways to pair four points in circular order, and exactly one of them produces crossing chords, so the pair crosses with probability $1/3$. By linearity over the $C(n, 2)$ pairs, the expectation is $C(n, 2)/3$.

Why the three pairings are equally likely. Name the four endpoints A, B, C, D in the order they sit around the circle. The four points were drawn independently, so which two of them happen to belong to the first chord is a uniformly random split of {A, B, C, D} into two pairs — and there are exactly three such splits. Only the split that joins opposite points (A–C with B–D) makes the chords cross.



Four random points in circular order A, B, C, D can be paired into two chords in three ways; exactly one pairing crosses, so $P(\text{cross}) = 1/3$.

> **ANSWER** The Reddit version uses **10 chords: $C(10, 2)/3 = 45/3 = 15$** intersections on average. (A simulation with 5 chords gives 3.31 against the exact $10/3$.)

C7. Expected sum of the last two uniforms drawn

Reported: *Reddit r/quantfinance and r/learnquant posts "HRT Quant Interview Question", images from the prep site quantprof.org ("#1030 Last Uniform II") — a prep site labels it HRT; a harder sequel to C4, which QuantVault lists as reported*

The question.

- Draw random numbers, each independent and uniform on $[0, 1]$, and add them up.
- Stop as soon as the running sum is greater than 1. Example: 0.4, 0.3, 0.5 stops after three draws (sum 1.2).
- What is the expected value of the sum of the **last two** numbers drawn ($0.3 + 0.5 = 0.8$ in the example)?

Answer. **$5/2 - e/2 \approx 1.141$.**

Solution. Write N for the number of draws and S(k) for the running sum after k draws. The last two numbers add up to S(N) − S(N−2), so find E[S(N)] and E[S(N−2)].

1. **$E[S(N)] = e/2$.** By Wald's identity, $E[S(N)] = E[N] \cdot E[\text{one draw}] = e \cdot \frac{1}{2}$ ($E[N] = e$ from C4).
2. **The key density.** For $k \geq 1$, the chance that S(k) is near t and still ≤ 1 has density $t^{k-1}/(k-1)!$ on $[0, 1]$ (C4's simplex volumes). Summed over all k this is $\mathbf{e}t$, plus the start $S(0) = 0$.
3. **$E[S(N-2)] = e - 5/2$.** $S(N-2) = t$ requires two more draws: one, x, keeping the sum ≤ 1 ($x \leq 1 - t$), then one pushing it past 1 (probability $t + x$). Integrating, $P(\text{both happen} \mid t) = \int (t + x)\, dx$ over $0 \leq x \leq 1 - t$ $= (1 - t^2)/2$, so $E[S(N-2)] = \int_0^1 t \cdot et \cdot (1 - t^2)/2\, dt = (1 - (6 - 2e))/2 = e - 5/2$. (The start $t = 0$ adds nothing to the mean.)

$$E[\text{last two}] = E[S_N] - E[S_{N-2}] = \frac{e}{2} - \left(e - \frac{5}{2}\right) = \frac{5}{2} - \frac{e}{2} \approx 1.1409$$

KEY Two neat by-products, worth quoting: the **last** number alone averages $2 - e/2 \approx 0.641$ (biased upward — big draws are more likely to be the one that crosses 1), while the **second-to-last** averages exactly $1/2$. Sum: $5/2 - e/2$. A 1,000,000-game simulation gives 1.1400; exact symbolic integration gives $5/2 - e/2$.

TRAP The tempting answer is 1 ("two uniforms, mean $\frac{1}{2}$ each"). The stopping rule biases the final draw: this is the same size-bias effect as the inspection paradox.

D · Symmetry and geometry

The key idea. Look for a transformation that swaps the event with its complement (relabel players, flip a coin, rotate the plane). If one exists, the probability is $1/2$ with no calculation. For two uniform variables, draw the unit square.

D1. n + 1 coins against n coins

Reported: *LinkJob 2025 HRT write-up; QuantVault, "More Heads With One Extra Coin", seen 2021*

The question.

- Player A flips $n + 1$ fair coins; player B flips n fair coins.
- What is the probability that A gets strictly more heads than B?

Answer. **$1/2$.**

Solution. Compare A's first n coins with B's n coins: either A is ahead, behind or tied, and "ahead" and "behind" are equally likely by symmetry (prob. x each, tie prob. $1 - 2x$). A wins if it was ahead (x), or tied and the extra coin is heads ($(1 - 2x)/2$). Total $x + \frac{1}{2} - x = \frac{1}{2}$. A quicker argument: A has one extra coin, so A has more heads than B exactly when A does **not** have more tails than B. "More heads" and "more tails" are complementary events, and swapping H and T shows they are equally likely — so each is $\frac{1}{2}$.

D2. Odd versus even number of heads

Reported: *Glassdoor, Business Development Programmer, Aug 2016; QuantVault, "Even vs Odd Heads", seen 2022*

The question.

- Flip 10 fair coins.
- Prove that the probability of an odd number of heads equals the probability of an even number of heads.

Answer. **Both are exactly 1/2.**

Proof. Condition on the first nine coins. Whatever their parity, the tenth coin flips the parity with probability $\frac{1}{2}$ and keeps it with probability $\frac{1}{2}$. So the final parity is a fair coin. (Algebraic version: $\Sigma_k\ (-1)^k\ C(10, k) = (1 - 1)^{10} = 0$, so the even and odd binomial sums are equal.)

FOLLOW-UP QUESTIONS

▶ **"What if the coin is biased, with P(heads) = p?"**
Expand both $(q + p)^n$ and $(q - p)^n$ with $q = 1 - p$. Adding them cancels every term with an odd number of heads and doubles the even ones, so $P(\text{even}) = ((q + p)^n + (q - p)^n)/2 = \mathbf{(1 + (1 - 2p)^n)/2}$. For a fair coin $(1 - 2p) = 0$ and this is exactly 1/2.

▶ **"Is an even number of heads evidence the coin is fair?"**
Hardly. For $p = 0.6$ and $n = 10$, $(1 - 2p)^n = (-0.2)^{10} \approx 10^{-7}$, so P(even) is still 0.5000001. The parity is almost a fair coin for **any** bias that is not extreme, so it carries almost no information about fairness.

D3. P(Y > 3X) for independent standard normals

Reported: *Glassdoor, Algorithm Developer, May 2025 (with the follow-up) and Oct 2018; QuantVault, seen 2020*

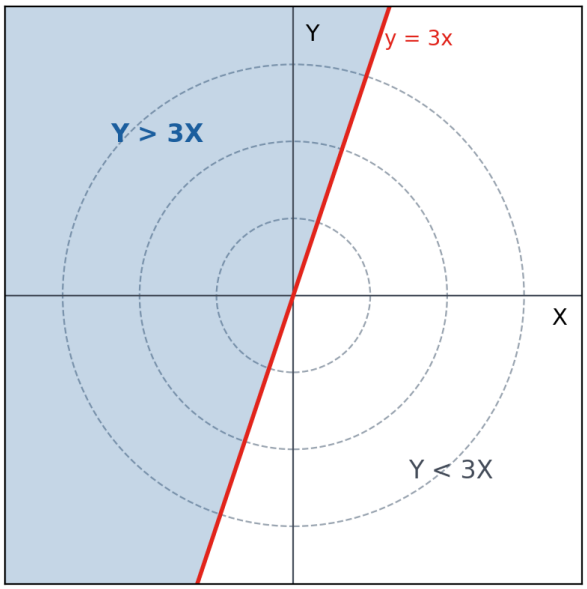
The question.

- X and Y are independent N(0, 1).
- (a) What is P(Y > 3X)?
- (b) Follow-up: given X > 0, what is P(Y > 3X)?

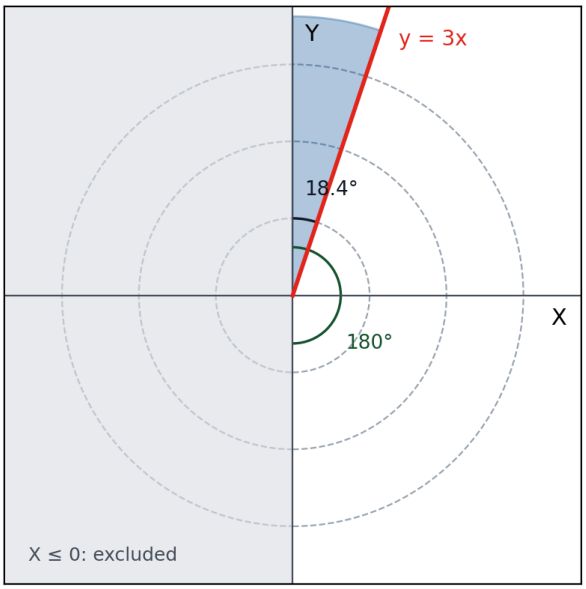
Answer. **(a) 1/2. (b) $(\pi/2 - \arctan 3)/\pi \approx 0.102$.**

Solution. (a) The pair (X, Y) is rotationally symmetric about the origin. The line $y = 3x$ passes through the origin and cuts the plane into two halves of equal probability. (b) Use polar angle: the angle θ of (X, Y) is uniform on $[0, 2\pi)$. $X > 0$ is half the circle. Within it, $Y > 3X$ is the wedge from $\arctan 3 \approx 71.57°$ up to 90°, an arc of 18.43°. So $P = 18.43°/180° \approx 0.1024$.

(a) any line through the origin halves the plane
P(Y > 3X) = 1/2



(b) given X > 0: the wedge from arctan 3 ≈ 71.6° to 90°
P = 18.4° / 180° ≈ 0.102



Dashed circles are contours of the joint density of two independent $N(0,1)$ variables: it depends only on distance from the origin, so every direction (angle) is equally likely. (a) The line $y = 3x$ splits the plane into two halves of equal probability. (b) Within the right half-plane (180° of angle), $Y > 3X$ is the 18.4° wedge between the line and the positive Y-axis.

> **TRAP** Do not try to integrate the bivariate normal. Whenever two iid normals meet a line through the origin, convert to an angle.

D4. Broken stick: can the three pieces form a triangle?

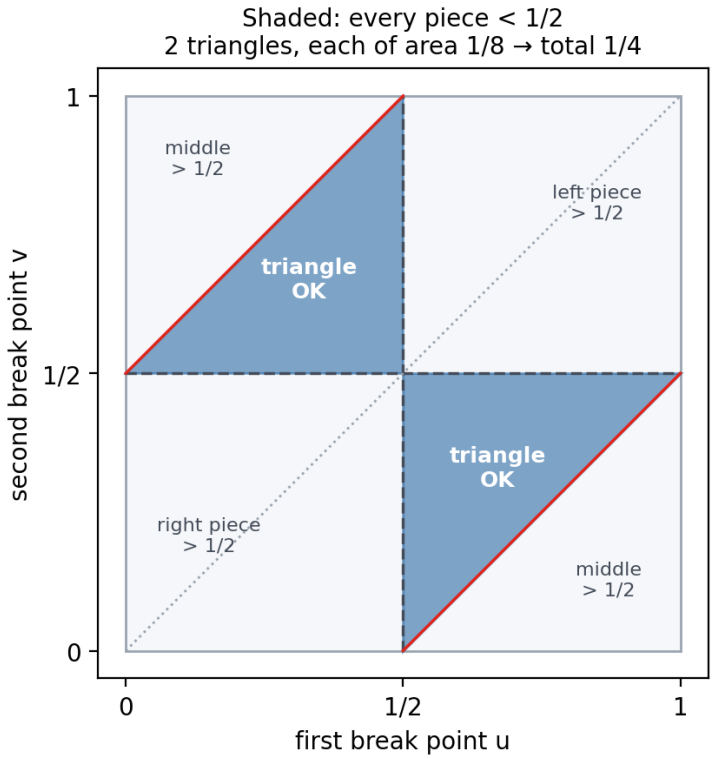
Reported: *LinkJob 2025 HRT write-up; QuantVault lists the harder "acute triangle" version, seen 2024*

The question.

- Take a stick of length 1. Choose two points on it independently and uniformly at random, and break the stick at both points, giving three pieces.
- What is the probability that the three pieces can form a triangle?

Answer. 1/4.

Solution. Three lengths form a triangle exactly when every piece is shorter than 1/2. With break points u, v drawn on the unit square, the region where all three pieces are below 1/2 is two small triangles of total area 1/4.



Each point (u, v) of the unit square is one way to break the stick. The pieces are $\min(u, v)$, $|u - v|$ and $1 - \max(u, v)$. The red and dashed lines mark where one piece reaches 1/2; only the two shaded triangles (area 1/8 each) have all three pieces below 1/2, so $P = 1/4$.

FOLLOW-UP QUESTIONS

▶ **"Expected length of the longest piece?"**
Answer: 11/18 ≈ 0.611. The cleanest route is the **uniform spacing** argument. Two random break points cut the stick into three pieces ("spacings") whose lengths are **exchangeable**: every ordering of the three pieces is equally likely, so no piece is special. Find the expected smallest, middle and largest piece in turn.

Smallest piece. All three pieces are at least x exactly when you can first set aside a length x for each piece and the two break points land in the remaining $1 - 3x$: $P(\text{smallest} \geq x) = (1 - 3x)^2$ for $0 \leq x \leq 1/3$. So E[smallest] = $\int_0^{(1/3)} (1 - 3x)^2\, dx$ = **1/9**. (In general, with n pieces it is $1/n^2$.)

Middle piece — peel off the smallest. Remove the smallest length from all three pieces. One piece becomes 0; the other two share the leftover length $1 - 3\cdot(\text{smallest})$, and they are again uniform spacings of it — two pieces this time. The smaller of two uniform pieces averages 1/4 of the length, so E[middle] = E[smallest] + $\frac{1}{4}\cdot$E[$1 - 3\cdot$smallest] = $1/9 + \frac{1}{4}\cdot(1 - 1/3) = 1/9 + 1/6$ = **5/18**.

Largest piece. The three expectations add to the whole stick: E[largest] = $1 - 1/9 - 5/18$ = **11/18**.

General formula. For n pieces, the k-th smallest has expectation $(1/n)\cdot(1/n + 1/(n-1) + \ldots + 1/(n-k+1))$; the largest is $(1/n)(1 + 1/2 + \ldots + 1/n)$. For n = 3: $(1/3)(11/6) = 11/18$. A 400,000-break simulation gives 0.611.

▶ **"Probability the triangle is acute?" — much harder: skip for now if not enough time**

$3 \ln 2 - 2 \approx 0.079$. A triangle is acute when each side^2 is less than the sum of the other two sides^2, which carves curved regions out of the square. Know the number and the condition; you are unlikely to be asked to derive it in a 25-minute slot.

D5. Romeo and Juliet: do they meet?

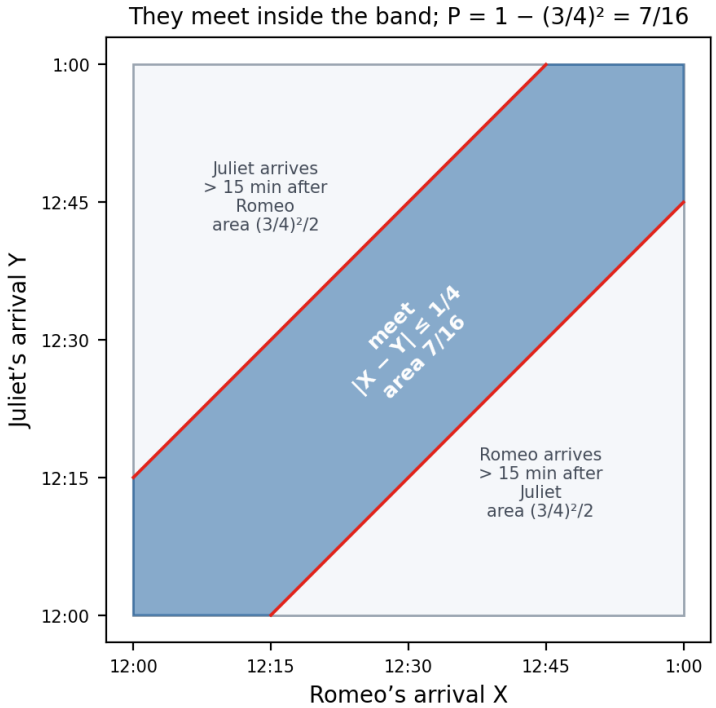
Reported: *Tradermath HRT guide, given as a representative HRT question (2026)*

The question.

- Two people agree to meet between 12:00 and 1:00. Each arrives at a time chosen uniformly at random within that hour, independently of the other.
- Whoever arrives first waits 15 minutes for the other, then leaves.
- What is the probability that they meet?

Answer. **7/16 ≈ 0.4375.**

Solution. Measure time in hours: they meet when $|X - Y| \leq 1/4$. On the unit square, the complement is two corner triangles with legs 3/4, of total area $(3/4)^2 = 9/16$. So $P = 1 - 9/16 = 7/16$.



Each point (X, Y) of the square is one pair of arrival times. They meet when the arrivals are within 15 minutes, the shaded band between the red lines. The two white corner triangles (legs 3/4, total area 9/16) are the misses, so $P(\text{meet}) = 1 - 9/16 = 7/16$.

E · The Central Limit Theorem (CLT) approximation question

E1. Sum of 100 dice versus heads in 600 coin flips

Reported: *Glassdoor: Algorithm Developer Jan 2022; Entry Level Quant Developer Sep 2022; Trading Algo Developer Apr 2022; Algorithm Developer Apr 2023; SWE Nov 2021 (variant). QuantVault, seen 2022 and 2024*

The question.

- X = the sum of the faces in 100 rolls of a fair six-sided die.
- Y = the number of heads in 600 flips of a fair coin.
- Estimate $P(X > Y)$ without a calculator.

Answer. **About 0.99** (exact value 0.99079).

Solution. Use the **Central Limit Theorem (CLT)**: a sum of many independent random variables is approximately normal, whatever the distribution of each one. X is a sum of 100 die rolls and Y a sum of 600 coin flips, so both — and their difference — are approximately normal.

- X: mean $100 \times 3.5 = 350$; variance $100 \times 35/12 \approx 291.7$.
- Y: mean $600 \times \frac{1}{2} = 300$; variance $600 \times \frac{1}{4} = 150$.
- D = X − Y: mean 50; variance ≈ 441.7 (independent, so variances add); standard deviation ≈ 21.0.

$$P(X > Y) = P(D \geq 1) \approx P\left(Z > \frac{0.5 - 50}{21.0}\right) = P(Z > -2.36) \approx 0.991$$

> **KEY** The interviewer is checking three things: you know the die variance 35/12 by heart ($E[X^2] = 91/6$, minus 3.5^2), you add variances of the difference rather than subtracting them, and you can turn 2.36 standard deviations into "about 99%".

FOLLOW-UP QUESTIONS

▶ **"What is P(the coins win), Y > X?"**
≈ **0.0081.** Use the same normal approximation in the other direction: $P(D \leq -1) \approx P(Z < (-0.5 - 50)/21.0) = P(Z < -2.40) \approx 0.008$. The three cases add up to 1: $P(X > Y) \approx 0.9908$, $P(Y > X) \approx 0.0081$, and a tie $P(X = Y) \approx 0.0011$.

▶ **"600 coin flips against 300 die rolls?"**
Now the die total has mean $300 \times 3.5 = 1{,}050$ against 300 heads — a gap of 750 with a standard deviation of only about 32 (variance 875 + 150). That is over 20 standard deviations, so P(coins win) is essentially **0**.

▶ **"Why the 0.5 (continuity correction)?"**
X − Y takes only whole-number values, so "D ≥ 1" is the same event as "D > 0.5"; using 0.5 places the cut-off halfway between 0 and 1 and makes the normal approximation more accurate. Here it moves the answer by well under 1%, so say so and move on.

F · Markov chains, games and optimal stopping

The key idea. Define one unknown per state ("probability of winning from here", "expected value from here"), write one equation per state by conditioning on the next step, and solve. For stopping games, work backwards from the last decision.

F1. Gambler's ruin

Reported: *LinkJob 2025 HRT phone interview (this exact wording); QuantVault, "Gambler's Ruin on a Fair Coin", seen Jun 2026*

The question.

- A gambler starts with $2. Each bet wins $1 with probability p and loses $1 with probability q = 1 − p.
- She stops on reaching $5 or $0.
- What is the probability she reaches $5?

Answer. $\mathbf{(1-(q/p)^2)/(1-(q/p)^5)}$ for $p \neq \frac{1}{2}$; **2/5** for $p = \frac{1}{2}$.

Solution. Let P_i be the probability of reaching \$5 when you currently hold \$i. The two barriers give the boundary values $\mathbf{P_0 = 0}$ (already broke) and $\mathbf{P_5 = 1}$ (already at the target). We want P_2.

1. **Condition on the next bet.** From \$i (strictly between the barriers) you win the bet with probability p and move to i + 1, or lose with probability q and move to i − 1. After that the game simply continues from the new amount, so:
$$P_i = p\,P_{i+1} + q\,P_{i-1} \qquad \text{for } 0 < i < N$$
2. **Look at the differences.** Write $P_i = (p + q)P_i$, so the equation becomes $p(P_{i+1} - P_i) = q(P_i - P_{i-1})$. With $d_i = P_i - P_{i-1}$ and $r = q/p$, each step multiplies the difference by r:
$$d_{i+1} = r\,d_i \quad \Longrightarrow \quad d_i = r^{i-1}d_1$$
3. **Add the differences up.** $P_i = P_0 + d_1 + \ldots + d_i = d_1(1 + r + \ldots + r^{i-1})$. This is a geometric series: $(1 - r^i)/(1 - r)$ when $r \neq 1$, and simply i when $r = 1$ (a fair coin).
4. **Use the top barrier to find d_1.** The top barrier ($P = 1$ at $i = N$) fixes $d_1 = 1/(1 + r + \ldots + r^{N-1})$. Dividing the two geometric sums, the factor $(1 - r)$ cancels:

$$P_i = (1 - r^i)/(1 - r^N) \text{ (biased, } r = q/p \neq 1) \quad P_i = i/N \text{ (fair, } p = q = \tfrac{1}{2})$$

Here $N = 5$ and $i = 2$: $P_2 = (1 - (q/p)^2)/(1 - (q/p)^5)$, and **2/5** for a fair coin.

General solution: two absorbing barriers

Start at i, stop at 0 or N, win each step with probability p, lose with $q = 1 - p$, and let $r = q/p$. (Equivalently: start at 0 with barriers a below and b above — set $i = a$, $N = a + b$.)

Quantity	Biased coin ($p \neq \frac{1}{2}$)	Fair coin ($p = \frac{1}{2}$)
P(reach N before 0)	$(1 - r^i)/(1 - r^N)$	i/N
P(ruin: reach 0 first)	$(r^i - r^N)/(1 - r^N)$	$(N - i)/N$
Expected number of bets	$i/(q - p) - (N/(q - p))\cdot(1 - r^i)/(1 - r^N)$	$i(N - i)$
Target very far away ($N \to \infty$)	P(ever ruined) = 1 if $p \leq q$; $= r^i = (q/p)^i$ if $p > q$	ruin is certain eventually

Check with this question ($i = 2$, $N = 5$, $p = 0.6$, $r = 2/3$): $P = (1 - 4/9)/(1 - 32/243) = 0.640$, and the expected number of bets is 5.995; both match a direct numerical solution of the equations. Fair case: 2/5 and $2 \times 3 = 6$ bets.

FOLLOW-UP QUESTIONS

▶ **"What is the answer for p = 0.6?"**
$(1 - (2/3)^2)/(1 - (2/3)^5) = (5/9)/(211/243) =$ **0.640** (a 100,000-game simulation gives 0.641). A small edge per bet compounds into a large edge in the outcome.

▶ **"How long does the fair game last on average?"**
Let D_i be the expected number of bets from fortune i. Then $D_i = 1 + \frac{1}{2}D_{i-1} + \frac{1}{2}D_{i+1}$ with $D_0 = D_5 = 0$. The solution is $\mathbf{D_i = i(N - i)}$ — from \$2 with target \$5: $2 \times 3 =$ **6 bets**.

▶ **"Why is the fair answer exactly i/N?"**
In a fair game your fortune is a martingale: its expected value never changes. At the end you hold either 5 (probability P) or 0, so $E[\text{final}] = 5P$ must equal the starting 2, giving $P = 2/5$.

F2. Winning a tennis game from deuce

Reported: *Glassdoor, Algorithm Developer phone screen, Sep 2025 ("probability of tennis match succeeding")*

The question.

- A tennis game is at deuce (level). From here, the first player to get two points ahead of the other wins the game.
- You win each point independently with probability p (and lose it with probability $q = 1 - p$).
- What is the probability that you win the game?

Answer. $\mathbf{p^2/(p^2 + q^2)}$, $q = 1 - p$ (0.692 for $p = 0.6$).

Solution. Look at points in pairs. Winning both (p^2) wins the game; losing both (q^2) loses it; splitting them ($2pq$) returns to deuce. So $W = p^2 + 2pq \cdot W$, giving $W = p^2/(1 - 2pq) = p^2/(p^2 + q^2)$.

> **TRAP** Since $p^2 + q^2 = 1 - 2pq$, both forms are equivalent; the second is easier to sanity-check ($p = \frac{1}{2}$ gives $\frac{1}{2}$).

F3. The dice game: roll up to three times, keep the last roll

Reported: *Glassdoor, Quant Researcher, Feb 2024 ("the dice brainteaser where you can play three times in a row"); a classic HRT shape — see P3 for the coding version*

The question.

- You may roll a fair die up to three times.
- After each roll you either stop and are paid that many dollars, or roll again (forfeiting that roll).
- You must accept the third roll if you get there. What is the game worth with optimal play, and what is the strategy?

Answer. **14/3 ≈ $4.67.** Keep a first roll of 5 or 6; keep a second roll of 4, 5 or 6.

Solution. Work backwards.

1. One roll left: worth 3.5.
2. Two rolls left: keep the roll if it beats 3.5 (i.e. 4, 5, 6), otherwise roll on: $(4 + 5 + 6 + 3 \times 3.5)/6 = 4.25$.
3. Three rolls left: keep the roll if it beats 4.25 (i.e. 5, 6), otherwise continue: $(5 + 6 + 4 \times 4.25)/6 = 28/6 = 14/3$.

FOLLOW-UP QUESTIONS

▶ **"What if you could roll four times?"**
One more backward step: keep a first roll that beats 14/3 (i.e. 5 or 6), otherwise continue: $(5 + 6 + 4 \times 14/3)/6 =$ **89/18 ≈ 4.94**. Each extra roll adds less, and the value approaches 6 because with many rolls you are almost sure to see a 6 eventually.

▶ **"How much would you pay to play the three-roll game?"**
Up to its expected value, **$4.67**, if you only care about the average. The interviewer may then ask about risk: the payoff ranges from 1 to 6, so a risk-averse player would pay a little less — a chance to show you separate expected value from risk.

▶ **"What if each extra roll costs c dollars?"**
Subtract c from the value of continuing: keep a roll if it beats (value of continuing − c). Thresholds fall, so you stop earlier.

F4. Spaghetti loops

Reported: *Glassdoor, Algorithm Developer phone interview, Oct 2016 (two versions)*

The question.

- A bowl holds n strands of spaghetti, so there are 2n loose ends.
- Repeatedly pick two of the remaining loose ends uniformly at random — they may belong to the same strand — and tie them together, until no loose ends remain.
- What is the expected number of closed loops formed?

Answer. $\mathbf{1 + 1/3 + 1/5 + \ldots + 1/(2n-1)}$.

Solution. Take any loose end and tie it to a random other end. With k strands there are $2k-1$ other ends, and exactly one of them is the other end of the same strand: probability $1/(2k-1)$ of closing a loop. Either way the number of strands drops from k to $k-1$. By linearity, sum over $k = n, n-1, \ldots, 1$. For $n = 3$: $1 + 1/3 + 1/5 \approx 1.53$.

F5. Spaghetti II: probability of one single big loop

Reported: *Reddit r/quantfinance and r/learnquant posts "HRT Quant Interview Question", images from the prep site quantprof.org ("#900 Noodle Loop II") — a prep site labels it HRT; the sequel to F4, which Glassdoor reports from an HRT phone interview*

The question.

- A bowl contains 100 separate noodles, so 200 loose ends.
- At each step, two of the remaining loose ends are chosen uniformly at random and tied together, until every end is tied.
- What is the probability that the final result is one single loop containing all 100 noodles?

Answer. $\mathbf{\prod_{k=2}^{100} (2k-2)/(2k-1) = (2\cdot4\cdot\ldots\cdot198)/(3\cdot5\cdot\ldots\cdot199) \approx 0.0887}$.

Solution. Use F4's observation: picking two random ends is the same as picking any one end and then a random partner among the other ends. With k strands left (2k ends), the partner is the same strand's other end with probability $1/(2k-1)$, which closes a loop early. For a single loop, this must **not** happen at any step until the very last strand ($k = 1$), which must close.

$$P(\text{one loop}) = \prod_{k=2}^{n}\left(1 - \frac{1}{2k-1}\right) = \prod_{k=2}^{n}\frac{2k-2}{2k-1} = \frac{4^n}{2n\binom{2n}{n}}$$

For $n = 100$ this is 0.08873. **Stirling's approximation** (see the box below), $C(2n, n) \approx 4^n/\sqrt{(\pi n)}$, gives the quick estimate $\sqrt{\pi}/(2\sqrt{n}) = 0.0886$ — say this in the interview, then the exact product. Small check: $n = 2$ gives $2/3$ (of the 3 ways to pair 4 ends, 2 join the two strands); a 200,000-run simulation with $n = 5$ gives 0.4076 against the exact 0.4063.

> **KEY** **Stirling's approximation:** $n! \approx \sqrt{(2\pi n)}\cdot(n/e)^n$, and its most useful consequence, $\mathbf{C(2n, n) \approx 4^n/\sqrt{(\pi n)}}$. Both get more accurate as n grows: off by about 1% at $n = 10$ and about 0.1% at $n = 100$.

Why it holds (short version, no full proof needed).

- **The $(n/e)^n$ part:** $\log n! = \log 1 + \log 2 + \ldots + \log n$, which is close to the area $\int_1^n \log x\,dx = n\log n - n + 1$. Exponentiating gives roughly $n^n e^{-n} = (n/e)^n$. The extra $\sqrt{(2\pi n)}$ factor is a correction from the curvature of $\log x$; you can quote it.
- **The binomial version, via the Central Limit Theorem — the easiest way to remember it:** $C(2n, n)/4^n$ is the probability of exactly n heads in 2n fair coin flips. The number of heads is approximately normal with mean n and variance $2n \cdot \frac{1}{4} = n/2$, so the chance of hitting the single most likely value is about the normal density at its peak, $1/\sqrt{(2\pi \cdot n/2)} = 1/\sqrt{(\pi n)}$. Multiply by 4^n: $C(2n, n) \approx 4^n/\sqrt{(\pi n)}$.
- **Applied here:** $P(\text{one loop}) = 4^n/(2n \cdot C(2n, n)) \approx 4^n/(2n \cdot 4^n/\sqrt{(\pi n)}) = \sqrt{(\pi n)}/(2n) = \sqrt{\pi}/(2\sqrt{n})$. For $n = 100$: $\sqrt{\pi}/20 \approx 0.0886$ (exact 0.0887).

FOLLOW-UP QUESTIONS

▶ **"And the expected number of loops with 100 noodles?"**
From F4: $1 + 1/3 + 1/5 + \ldots + 1/199 \approx$ **3.28**. For large n this is about $\frac{1}{2}\ln n + \ln 2 + \gamma/2$ ($\gamma \approx 0.577$), which grows very slowly.

▶ **"How can the average be only about 3 loops while a single loop has only a 9% chance?"**
Because the number of loops is spread over 1, 2, 3, 4, 5 ... with no single value dominant. A 9% chance for exactly one loop is consistent with an average of about 3 — the distribution is just wide.

G · Statistics, estimators and "now simulate it"

The key idea. These are the questions the July 2026 sophomore and the 2026 Algo Developer reports describe — a statistics derivation followed by a simulation that checks it. Bias is E[estimator] − true value; variance is the spread of the estimator over repeated samples; mean squared error = bias^2 + variance.

G1. Estimate θ from Uniform$(0, \theta)$ data: bias and variance

Reported: *Glassdoor, Sophomore intern, Jul 2026 ("coming up with an estimator and then coding it to check variance and bias"); Algorithm Developer, Feb 2026 ("derive the probability of some order statistic"); QuantVault, "MLE for Uniform Distribution", seen Aug 2026*

The question.

- You observe $X_1, \ldots, X_n$ independent Uniform$(0, \theta)$, with θ unknown.
- Propose an estimator of θ. Is it biased? What is its variance?
- Then: write code to check your answers by simulation.

Answer. Natural estimators, compared in a worked example. θ **= 10** is the true (in real life unknown) upper limit we pretend to know so we can measure each estimator's error, and **n = 10** is the number of observations in each sample. The formulas hold for any θ and n; the numbers plug in $\theta = 10$, $n = 10$.

Estimator	Bias	Variance	Comment
$M = \max X_i$ (the MLE)	$-\theta/(n+1) = -0.909$	$n\theta^2/((n+1)^2(n+2)) = 0.689$	Biased low: the maximum can never exceed θ.
$(n+1)/n \cdot M$	0	$\theta^2/(n(n+2)) = 0.833$	Unbiased, and the **best unbiased** estimator: lowest variance of any unbiased estimator.
$2 \cdot$ (sample mean)	0	$\theta^2/(3n) = 3.333$	Unbiased (method of moments) but four times noisier.
$(n+2)/(n+1) \cdot M$	$-\theta/(n+1)^2 = -0.083$	0.820 (MSE $\theta^2/(n+1)^2 = 0.826$)	Slightly biased, but the **lowest mean squared error** of all estimators of the form $c \cdot M$.

Solution. $P(M \le x) = (x/\theta)^n$, so M has density $n\,x^{n-1}/\theta^n$ on $[0, \theta]$, giving $E[M] = n\theta/(n+1)$ and $E[M^2] = n\theta^2/(n+2)$. Variance = $E[M^2] - E[M]^2$. Scaling by $(n+1)/n$ removes the bias. For $2\cdot\bar{X}$: $\mathrm{Var}(X_i) = \theta^2/12$, so $\mathrm{Var}(2\bar{X}) = 4\cdot\theta^2/(12n) = \theta^2/(3n)$.

Is $(n+1)/n \cdot M$ the best estimator? It depends on what "best" means — say this distinction out loud.

- **Best among unbiased estimators: yes.** The maximum M contains all the information the sample has about θ (it is a sufficient statistic: once you know the largest value, the others are just uniform below it and tell you nothing more). A standard theorem (Lehmann–Scheffé) then says the unbiased estimator built from M, namely $(n+1)/n \cdot M$, has the smallest variance of **every** unbiased estimator. So it beats $2 \cdot \bar{X}$ and anything else that is unbiased.

- **Best by mean squared error (MSE = bias² + variance): no, slightly.** If you allow a little bias, shrink M by a different factor c and choose c to minimise $E[(cM - \theta)^2]$. The minimum is at $c = E[M]\cdot\theta \,/\, E[M^2] = (n+2)/(n+1)$, giving $MSE = \theta^2/(n+1)^2$. For $\theta = 10$, $n = 10$ that is $100/121 =$ **0.826**, just below the unbiased estimator's 0.833. The gain is tiny: it trades a small bias (-0.083) for lower variance.
- **In an interview:** propose M first (the natural guess), point out it is biased low, correct it to $(n+1)/n \cdot M$, and if asked "can you do better?" mention the MSE trade-off. That sequence shows you understand bias, variance and MSE separately.

> **KEY** The simulation that checks this is P2 in Part 4: it reproduced every number in the table (bias −0.906, variances 0.686, 0.830, 3.334). Say before running it what you expect to see — that is the point of the exercise.

G2. Why divide by n − 1 in the sample variance?

Reported: *Quantt HRT guide lists "estimator of a quantity and to tell if the estimator is biased or not" among reported phone-screen questions*

The question.

- $X_1, \ldots, X_n$ are independent with the same mean μ and variance σ^2, and $\bar{X}$ is their average.
- Is $S^2 = (1/n)\,\Sigma\,(X_i - \bar{X})^2$ an unbiased estimator of σ^2? If not, how do you fix it?

Answer. **No: $E[S^2] = (n-1)/n \cdot \sigma^2$.** Dividing by $n-1$ instead removes the bias.

Solution. $\Sigma\,(X_i - \bar{X})^2 = \Sigma\,(X_i - \mu)^2 - n(\bar{X} - \mu)^2$. Taking expectations: $n\sigma^2 - n\cdot(\sigma^2/n) = (n-1)\sigma^2$. The deviations are measured from $\bar{X}$, which sits closer to the data than the true mean, so they are systematically too small.

G3. Generate two correlated normals

Reported: *Glassdoor, Quant Researcher, Oct 2024*

The question.

- You can generate independent $N(0,1)$ variables Z_1, Z_2.
- Produce X and Y, each $N(0,1)$, with correlation ρ.

Answer. $\mathbf{X = Z_1,\ Y = \rho Z_1 + \sqrt{(1-\rho^2)}\, Z_2}$.

Solution. $\mathrm{Var}(Y) = \rho^2 + (1-\rho^2) = 1$, and $\mathrm{Cov}(X, Y) = \rho\cdot\mathrm{Var}(Z_1) = \rho$. This is the 2×2 Cholesky factor of the correlation matrix; in code, np.random.default_rng().multivariate_normal does the same.

G4. Can he really taste the colour of M&Ms?

Reported: *Glassdoor, Algorithm Developer, Dec 2020; QuantVault, "Hypothesis Test for M&M Color Identification", seen 2025*

The question.

- Someone claims he can tell an M&M's colour by taste alone. There are 5 colours, equally common.
- Blindfolded, he is given 10 M&Ms and names the colour correctly 5 times out of 10.
- Should you believe him? (Quantify how likely this is by pure guessing.)

Answer. **Weak evidence: p-value ≈ 0.033.**

Solution. Null hypothesis: he is guessing, so the number correct is Binomial(10, 1/5) with mean 2. P(at least 5 correct) $= \Sigma_{k=5}^{10}\, C(10,k)(0.2)^k(0.8)^{10-k} \approx 0.033$. That is below 5% but only just — with a single test of 10 trials, ask for more trials before believing him.

FOLLOW-UP QUESTIONS

▶ **"What if he got 4 out of 10?"**
P(at least 4 correct | guessing) $= \Sigma_{k=4}^{10}\, C(10,k)(0.2)^k(0.8)^{10-k} \approx$ **0.12** — a 12% chance by pure luck, so not convincing.

▶ **"How many trials would you want?"**

This is a question about **power**: the chance the test detects real skill. If his true hit rate were 50%, a 5%-level test with 10 trials (reject if 5 or more correct) catches him only **62%** of the time. With 25 trials (reject if 9 or more correct) the power rises to about **95%** while the false-alarm rate stays below 5%.

Part 4 — The programming exercise: reported live-coding problems, solved

Twenty-seven problems. P1–P4 are the "derive, then simulate or compute" family that HRT intern and early-career rounds report most; P5–P14 are self-contained live-coding problems HRT candidates have reported in interviews, P15 is the most-reported HRT online-assessment problem, and P16–P27 are newer reports, mostly from 2026 technical screens. Every problem ends with a step-by-step explanation of the code, its complexity, and how to test it live. All code is Python 3, was run before being written here, and passes randomised tests against brute force.

> **KEY** For an hour that is half probability, the coding part is likely to be short — one problem of 20–25 minutes, possibly built on the probability question you just solved. Spend most of your practice on P1–P4, and use P5–P27 to rehearse the habits: state the plan, write the simple version, run it, then improve.

P1. A reusable Monte Carlo harness — type this from memory

Reported: *The pattern behind the May 2026, Jan 2026 and Nov 2025 reports: "implement a tiny simulation in Python", "run simulation to check your intuition", "debug the corresponding simulation code"*

The task.

- Given any random experiment that returns a number (a count, a payoff, or 1/0 for an event), estimate its expected value.
- Report the estimate with a 95% confidence interval, so you can say whether it agrees with your hand-derived answer.

The approach. Run the experiment n times and keep a running sum and sum of squares. The estimate is the sample mean; its standard error is $\sqrt{(\text{variance}/n)}$; 1.96 standard errors either side is a 95% interval. Seed the generator so the run is reproducible.

```python
import random, math

def monte_carlo(trial, n=200_000, seed=0):
    """Run trial() n times; return (estimate, 95% half-width)."""
    random.seed(seed)
    s = s2 = 0.0
    for _ in range(n):
        x = trial()
        s += x
        s2 += x * x
    mean = s / n
    var = s2 / n - mean * mean
    return mean, 1.96 * math.sqrt(var / n)

def rolls_until_sum7():
    c = 0
    while True:
        c += 1
        if random.randint(1, 6) + random.randint(1, 6) == 7:
            return c
```

```python
if __name__ == "__main__":
    est, hw = monte_carlo(rolls_until_sum7)
    print(f"{est:.3f} +/- {hw:.3f}   (exact 6)")
# output:  5.983 +/- 0.024   (exact 6)
```

> **TIP** The interval is what makes this convincing. Saying "5.98 $\pm$ 0.02, consistent with 6" sounds like a researcher; saying "about 6" does not. If your hand answer lies outside the interval, one of the two is wrong — say which one you would check first.

> **TRAP** The error shrinks like $1/\sqrt{n}$: four times the trials only halves the error. And a simulation cannot confirm an infinite mean (C1) — the running average keeps drifting up, which is itself the evidence.

How the code works, step by step.

1. **trial** is any function that returns a number. To estimate a probability, make it return 1 when the event happens and 0 otherwise — the average of 0/1 values is the probability.
2. **s and s2** hold the running sum and running sum of squares, so nothing is stored per trial: memory stays constant however large n is.
3. **mean = s / n** is the estimate. **var = s2 / n − mean²** is the spread of a single trial's result ($E[X^2] - E[X]^2$).
4. **1.96 · √(var / n)** is the 95% half-width: by the central limit theorem the average of n trials is roughly normal with standard deviation $\sqrt{\text{var} / n}$, and 95% of a normal lies within 1.96 standard deviations.
5. **random.seed(seed)** makes the run reproducible, so you and the interviewer see the same numbers when you re-run it.
6. **rolls_until_sum7** is the experiment itself: a counter, a loop, and a return as soon as the two dice sum to 7.

Complexity. $O(n \times \text{cost of one trial})$ time, $O(1)$ memory. 200,000 trials of a short experiment take a second or two in Python.

How to test it live. First run it on something whose answer you know for certain, e.g. trial = lambda: random.random() < 0.3 should give 0.300 ± 0.002. Then run the real experiment.

P2. Check an estimator's bias and variance by simulation

Reported: *Glassdoor, Sophomore intern, Jul 2026 — the closest reported round to WiTTI in level and date*

The task.

- For Uniform$(0, \theta)$ data (question G1), simulate many samples of size n.
- For each estimator, report its bias, variance and mean squared error, and compare them with the formulas.

The approach. Draw a reps × n array in one numpy call, compute each estimator along axis 1, then take the mean and variance across the reps. Vectorised numpy makes 100,000 samples instant.

```python
import numpy as np

def compare_estimators(theta=10.0, n=10, reps=100_000, seed=0):
    rng = np.random.default_rng(seed)
    x = rng.uniform(0, theta, size=(reps, n))
    est = {
        "max (MLE)":        x.max(axis=1),
        "(n+1)/n * max":    (n + 1) / n * x.max(axis=1),
        "2 * mean":         2 * x.mean(axis=1),
    }
    for name, e in est.items():
        bias = e.mean() - theta
        var = e.var()
        mse = ((e - theta) ** 2).mean()
        print(f"{name:16s} bias={bias:+.4f}  var={var:.4f}  mse={mse:.4f}")
```

```
# output (theta = 10, n = 10):
# max (MLE)        bias=-0.9056  var=0.6860  mse=1.5061
# (n+1)/n * max    bias=+0.0039  var=0.8301  mse=0.8301
# 2 * mean         bias=+0.0032  var=3.3342  mse=3.3342
```

> **SAY THIS** "The MLE has the smallest variance but its bias makes its MSE worse than the corrected estimator; the moment estimator is unbiased but four times noisier. All three match the formulas to within simulation error."

FOLLOW-UP Likely next questions: "What happens as n grows?" (the max-based estimators' variance falls like $1/n^2$, the mean-based one like $1/n$); "Which would you use?" ($(n+1)/n \cdot \max$ — lowest MSE among unbiased ones); "How many reps do you need?" (enough that the standard error of the bias estimate, $\sqrt{}(\text{var}/\text{reps})$, is well below the bias you are trying to detect).

How the code works, step by step.

1. **rng.uniform(0, θ, size=(reps, n))** draws a whole table at once: each of the reps rows is one data set of size n.
2. **x.max(axis=1)** takes the maximum along each row, giving reps estimates of θ in one vectorised call (no Python loop).
3. For each estimator, **e.mean() − θ** is the bias, **e.var()** the variance and **((e − θ)²).mean()** the mean squared error. The MSE should equal bias^2 + variance — a free consistency check.
4. Scaling the max by $(n+1)/n$ removes its bias; doubling the sample mean uses $E[X] = \theta/2$.

Complexity. $O(\text{reps} \times n)$ arithmetic, all inside numpy: 10^6 numbers take milliseconds.

How to test it live. Try n = 1: the max is just the single draw, so its bias must be $-\theta/2 = -5$. Then change n and check the variance shrinks like $1/n^2$ for the max and $1/n$ for the mean.

P3. Find the optimal strategy for a dice game, then confirm it by simulation

Reported: *Glassdoor: Quantitative Researcher, Nov 2025 ("develop an optimal strategy in a game with dice, then live coding to validate the optimal strategy"); Algorithm Developer intern, Apr 2025 ("a game where you need to code the optimal strategy"); Tradermath describes this as a common HRT format*

The task.

- You may roll a fair die up to max_rolls times. After each roll, stop and take that value, or roll again.
- Compute the value of the game with optimal play and the threshold to beat at each roll.
- Then simulate the strategy to confirm the value.

The approach. Backward induction (question F3) as a loop: the value with one roll left is 3.5; with one more roll available, keep any roll above the current value, otherwise continue. Use Fraction for exact answers. The simulation then plays the thresholds directly.

```python
import random
from fractions import Fraction

def dice_game_value(max_rolls, faces=6):
    """You may roll up to max_rolls times; after each roll you either keep
    that value (game ends) or roll again. Returns (value, thresholds)."""
    v = Fraction(sum(range(1, faces + 1)), faces)   # value with 1 roll left
    thresholds = []
    for _ in range(max_rolls - 1):
        thresholds.append(v)                        # keep a roll only if it beats v
        v = Fraction(sum(max(k, v) for k in range(1, faces + 1)), faces)
    return v, thresholds[::-1]

def simulate(max_rolls, trials=300_000, faces=6):
    _, th = dice_game_value(max_rolls, faces)
    total = 0
    for _ in range(trials):
        for r in range(max_rolls):
            x = random.randint(1, faces)
            if r == max_rolls - 1 or x > th[r]:
                total += x
                break
    return total / trials
```

```
# output: rolls, exact value, keep-if-above thresholds, simulated value
# 1 7/2   3.5    []                  3.506
# 2 17/4  4.25   [3.5]               4.25
# 3 14/3  4.667  [4.25, 3.5]         4.666
# 4 89/18 4.944  [4.667, 4.25, 3.5]  4.946
```

> **FOLLOW-UP** Typical extensions: a cost per roll (subtract it inside the max); an n-sided die (pass faces); "two players, the higher final roll wins" — then the opponent's strategy changes yours, and the threshold is no longer the expected value.

How the code works, step by step.

1. **v** is the value of the game with the rolls still available. With one roll left you must accept it, so v starts at the average face, 7/2.
2. Each loop iteration adds one more roll in front. On that roll you see face k and choose the better of keeping k or continuing (worth v): the new value is the average of **max(k, v)** over the six faces.
3. The **threshold** for a roll is the value of continuing, so it is recorded **before** v is updated. The list is built from the last roll backwards, so it is reversed at the end to put roll 1 first.
4. Fraction keeps the answers exact (14/3 rather than 4.666...).
5. **simulate** plays the strategy: on roll r it keeps x if this is the last roll or x beats th[r]. Ties (x equal to the threshold) can go either way without changing the value.

Complexity. O(max_rolls × faces) for the exact value; O(trials × max_rolls) for the simulation.

How to test it live. max_rolls = 1 must give 3.5 and max_rolls = 2 must give 4.25 — both easy by hand. The simulated values should match to two decimals.

P4. Optimal strategy for a simplified blackjack

Reported: *Glassdoor: Algorithm Developer intern, Apr 2025 ("Code an optimal strategy for a simplified version of blackjack (45 mins live coding)"); Algorithm Developer intern, Oct 2023 ("code up a blackjack solver"). Related: Quant Trader intern, May 2026, derive the optimal Baccarat strategy*

The task (one reasonable set of simplified rules — confirm the interviewer's).

- Infinite deck: each card is 1–9 with probability 1/13 each, or 10 with probability 4/13. An ace counts as 1 only.
- The player gets two cards; the dealer shows one card.
- The player hits (takes a card) or stands; going over 21 loses immediately.
- The dealer then draws until reaching 17 or more; over 21 is a dealer bust.
- Win +1, lose −1, tie 0. Find the optimal hit/stand rule and the game's expected value.

The approach. Two memoised recursions. dealer_dist(total) gives the probability of each final dealer total (22 = bust) from a starting total. best(player, up) compares standing — score the player's total against that distribution — with hitting — average best() over the next card. The states are only player totals 2–21 × 10 up-cards, so recursion is instant. Then a simulation plays the computed strategy to confirm the value.

```
"""Simplified blackjack: infinite deck, card values 1..10 (10 four times as likely,
ace counts 1 only). Dealer hits below 17. Player busts over 21 -> loses.
Win +1, lose -1, push 0. Find the optimal hit/stand rule and its value."""
import random
from functools import lru_cache

CARD_P = {v: 1/13 for v in range(1, 10)}
CARD_P[10] = 4/13

@lru_cache(None)
def dealer_dist(total):
    """Distribution of the dealer's final total (22 = bust) starting from `total`."""
    if total > 21:
```

```
        return {22: 1.0}
    if total >= 17:
        return {total: 1.0}
    out = {}
    for c, p in CARD_P.items():
        for t, q in dealer_dist(total + c).items():
            out[t] = out.get(t, 0) + p * q
    return out

def ev_stand(player, up):
    ev = 0.0
    for t, q in dealer_dist(up).items():
        if t == 22 or player > t:
            ev += q
        elif player < t:
            ev -= q
    return ev

@lru_cache(None)
def best(player, up):
    """(EV, action) for a player total, dealer up-card value `up`."""
    if player > 21:
        return -1.0, "bust"
    stand = ev_stand(player, up)
    hit = sum(p * best(player + c, up)[0] for c, p in CARD_P.items())
    return (hit, "H") if hit > stand else (stand, "S")

def game_value():
    """Value before any card is dealt: player gets 2 cards, dealer shows 1."""
    v = 0.0
    for a, pa in CARD_P.items():
        for b, pb in CARD_P.items():
            for u, pu in CARD_P.items():
                v += pa * pb * pu * best(a + b, u)[0]
    return v

def draw():
    r = random.random(); acc = 0
    for c, p in CARD_P.items():
        acc += p
        if r < acc:
            return c
    return 10

def simulate(trials=300_000):
    tot = 0
    for _ in range(trials):
        pl = draw() + draw(); up = draw()
        while pl <= 21 and best(pl, up)[1] == "H":
            pl += draw()
        if pl > 21:
            tot -= 1; continue
        d = up
        while d < 17:
            d += draw()
        tot += 1 if (d > 21 or pl > d) else (-1 if pl < d else 0)
    return tot / trials
```

```
# output:
# dealer shows  1: stand on 14+      dealer shows  6: stand on 12+
# dealer shows  2: stand on 13+      dealer shows  7: stand on 17+
# dealer shows  3: stand on 13+      dealer shows  8: stand on 17+
# dealer shows  4: stand on 12+      dealer shows  9: stand on 17+
# dealer shows  5: stand on 12+      dealer shows 10: stand on 16+
# exact value -0.0574 simulated -0.0566
```

SAY THIS Explain the shape of the answer: against a weak dealer card (4–6) you stand early, because the dealer is likely to bust; against 7–9 you hit up to 16, because the dealer will usually make 17+. That pattern matching real basic strategy is a sanity check the interviewer will like.

TRAP Agree the rules first — ace as 1 or 11, dealer hits soft 17 or not, ties. Each changes the code. Adding soft aces means tracking (total, has_usable_ace) as the state: mention it as the next step rather than starting with it.

How the code works, step by step.

1. **CARD_P** maps each card value to its probability (nine values at 1/13 plus 10 at 4/13; they sum to 1). An infinite deck means every draw has the same probabilities.
2. **dealer_dist(total)** returns the probability of each final dealer total. Base cases: over 21 is a bust (stored as 22); 17–21 means the dealer stands. Otherwise it averages, over the next card, the distribution from total + card. **@lru_cache** remembers every result, so each total is computed once.
3. **ev_stand(player, up)** scores standing: add q for every dealer outcome the player beats (including a dealer bust), subtract q for every outcome that beats the player, ignore ties.
4. **best(player, up)** returns the better of standing and hitting. Hitting is worth the average of best(player + card) over the next card — a recursion that always ends because totals only go up and anything over 21 returns −1.
5. **game_value** averages best() over the player's first two cards and the dealer's up-card.
6. **simulate** plays complete hands with the strategy best() chose and checks that the average result matches the exact value — the "derive, then validate" habit HRT looks for.

Complexity. About 20 player totals × 10 up-cards = 200 states, each looking at 10 cards: instant. The simulation is O(trials).

How to test it live. Check that dealer_dist(up) sums to 1 for every up-card, and that best(21, up) always says stand. Then compare exact and simulated values.

P5. Hotter or colder: find a secret number

Reported: *Glassdoor, Junior Algo Developer technical round, Nov 2023 ("Guess a random integer from 0–1000. Each time respond if the current guess is closer/further to the previous guess"); QuantVault, "Hotter/Colder Guessing Game", seen 2024*

The task.

- A secret integer s lies in [0, 1000].
- After your first guess, each new guess is answered "hotter" (closer to s than your previous guess), "colder" (farther), or "same" (equally far).
- Write a strategy that always finds s, and count the guesses it needs.

The approach. Guess the two ends of the current range, lo then hi. The answer to hi says which side of the midpoint (lo + hi)/2 the secret is on ("same" means it is the midpoint). That halves the range with two guesses: about $2 \cdot \log_2(1001) \approx 20$ in the worst case.

```
"""Secret integer s in [lo, hi]. After your first guess, every new guess g is
answered 'hotter' (closer to s than the previous guess), 'colder' (farther) or
'same' (equally far).  Find s."""

def make_oracle(secret):
    prev = [None]
    def guess(g):
        p, prev[0] = prev[0], g
        if p is None:
            return None
        dp, dg = abs(p - secret), abs(g - secret)
        return "hotter" if dg < dp else "colder" if dg > dp else "same"
    return guess
```

```python
def find(lo, hi, guess):
    count = 0
    while lo < hi:
        guess(lo); r = guess(hi); count += 2       # compare the two endpoints
        mid2 = lo + hi                             # the tie point is (lo+hi)/2
        if r == "same":
            return mid2 // 2, count                # lo+hi even: secret is the midpoint
        if r == "hotter":                          # secret is nearer hi
            lo = mid2 // 2 + 1
        else:                                      # secret is nearer lo
            hi = (mid2 - 1) // 2
    return lo, count
```

```python
# test: every secret 0..1000 is found; worst case 18 guesses
```

> **FOLLOW-UP** "Can you do better?" Yes: reuse the previous guess as one end of the next comparison by reflecting guesses across the midpoint, which approaches $\log_2 n + O(1)$ guesses. Explain the idea; the two-guess version is the safe one to code under time pressure.

How the code works, step by step.

1. **make_oracle(secret)** builds the judge. It stores the previous guess in a one-element list so the inner function can update it, and answers by comparing distances to the secret.
2. In **find**, guessing lo and then hi makes the second answer a comparison between the two ends: "hotter" means the secret is closer to hi than to lo.
3. The two ends are equally far from the secret exactly at the midpoint $(lo + hi)/2$, so "same" means the secret is that midpoint (possible only when lo + hi is even).
4. "Hotter" means the secret is strictly above the midpoint: **lo = (lo + hi)//2 + 1**. "Colder" means strictly below: **hi = (lo + hi − 1)//2**. Both formulas handle odd and even lo + hi.
5. The loop ends when lo == hi: one candidate left, which must be the secret.

Complexity. Two guesses halve the range, so about $2 \cdot \log_2(1001) \approx 20$ guesses; the test found a worst case of 18.

How to test it live. Loop over every secret 0–1000 and assert the answer — the full check runs in under a second.

P6. A tic-tac-toe-style game engine (k in a row)

Reported: *Glassdoor: Software Engineer, Oct 2025 ("write a program for a tic-tac-toe-like game"); Software Developer, Sep 2025 ("implement a game that is kinda like tic tac toe"); SWE intern, Oct 2025 ("Lines and Squares"); Trading Algo Developer, Sep 2024 ("simulate turns on a board and output the final results")*

The task.

- Two players alternately place a mark on an R × C board.
- The first to own k consecutive cells in a row, column or diagonal wins.
- Implement play(r, c), which rejects illegal moves and reports the winner, plus a draw check.

The approach. A class holding the board, whose turn it is and the winner. After each move, check only the four lines through that cell: count matching marks outward in both directions, and the move wins if 1 + left + right ≥ k. That is O(k) per move instead of rescanning the whole board.

```python
class KInARow:
    """Two players alternately drop a mark on an R x C board; the first to own
    k consecutive cells in a row, column or diagonal wins (tic-tac-toe: 3,3,3)."""
    DIRS = [(0, 1), (1, 0), (1, 1), (1, -1)]

    def __init__(self, rows=3, cols=3, k=3):
        self.R, self.C, self.k = rows, cols, k
        self.board = [[None] * cols for _ in range(rows)]
        self.turn, self.moves, self.winner = 0, 0, None
```

```
    def play(self, r, c):
        if self.winner is not None:
            raise ValueError("game over")
        if not (0 <= r < self.R and 0 <= c < self.C) or self.board[r][c] is not None:
            raise ValueError("illegal move")
        p = self.turn
        self.board[r][c] = p
        self.moves += 1
        if self._wins(r, c, p):
            self.winner = p
        self.turn ^= 1
        return self.winner

    def _count(self, r, c, dr, dc, p):
        n = 0
        r, c = r + dr, c + dc
        while 0 <= r < self.R and 0 <= c < self.C and self.board[r][c] == p:
            n += 1; r += dr; c += dc
        return n

    def _wins(self, r, c, p):
        # only lines through the last move can have changed: O(k) per move
        return any(1 + self._count(r, c, dr, dc, p) + self._count(r, c, -dr, -dc, p) >= self.k
                   for dr, dc in self.DIRS)

    def is_draw(self):
        return self.winner is None and self.moves == self.R * self.C
```

```
# test: 3,000 random games on random board sizes; the O(k) check agrees with a full-board scan
```

> **TIP** This is a design-and-correctness problem, not an algorithms one. Interviewers look for validation of input, a clear separation between state and rules, and a test you actually run. Say how you would extend it (gravity for Connect Four: a move picks only a column).

How the code works, step by step.

1. The board stores None for empty and 0/1 for the players; **turn ^= 1** flips between 0 and 1.
2. **play** validates everything (game over, in bounds, cell empty) **before** changing any state, so an illegal move never leaves the board half-updated.
3. **_count(r, c, dr, dc, p)** walks outward from the new mark in one direction and counts consecutive cells owned by p.
4. **_wins** checks the four line directions (horizontal, vertical, two diagonals). For each, the line through the new mark has 1 + left count + right count cells; reaching k wins.
5. **is_draw** is a full board with no winner.

Complexity. O(k) per move instead of O(R·C) for a full rescan; O(R·C) memory for the board.

How to test it live. Play out a known tic-tac-toe win and a known draw by hand, then compare against a full-board scan on random games (as in the test).

P7. Maximum-profit contiguous run of trades (Kadane), and the one-deletion variant

Reported: *DEV Community write-up of an HRT video interview, Nov 2025 (with follow-ups on $O(n)$ vs $O(n^2)$, negative values, millions of elements); QuantVault, "Maximum Subarray Sum With One Deletion", seen Jun 2026; Medium HRT guide*

The task.

- (a) Given a list of trade P&Ls (positive or negative), return the largest sum of a contiguous non-empty block.
- (b) Same, but you may delete at most one element from the block (the block must stay non-empty).

The approach. (a) Keep the best sum of a block ending at the current element: either extend the previous block or restart here. (b) Keep two such values — with no deletion so far, and with one deletion used. Deleting the current element means taking the no-deletion value from the previous step.

```python
def max_subarray(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)        # extend the run or restart at x
        best = max(best, cur)
    return best

def max_subarray_one_deletion(a):
    keep = a[0]                      # best sum ending here, nothing deleted
    dele = float("-inf")             # best sum ending here, one element deleted
    best = a[0]
    for x in a[1:]:
        dele = max(dele + x, keep)   # delete x (take keep) or extend an earlier deletion
        keep = max(x, keep + x)
        best = max(best, keep, dele)
    return best
```

```python
# max_subarray([-2,1,-3,4,-1,2,1,-5,4]) = 6;  one deletion on [1,-2,0,3] = 4
# both checked against brute force on 3,000 random arrays
```

> **TRAP** If every value is negative the answer is the largest single element, not 0 — initialise with a[0], not 0. For the follow-up "millions of elements": the algorithm is one pass with O(1) memory, so it can run on a stream.

How the code works, step by step.

1. (a) **cur** is the best sum of a block that ends exactly at the current element. Either the block is just x, or it extends the best block ending one step earlier: cur = max(x, cur + x).
2. **best** is the maximum of cur over all positions — the best block ending anywhere.
3. (b) **keep** is (a)'s cur. **dele** is the best sum of a block ending here with one element already deleted. Either the deletion happened earlier (dele + x) or we delete x itself, leaving the best undeleted block that ended just before (the previous keep).
4. Update **dele before keep**, because dele needs the previous keep. Starting dele at $-\infty$ stops a "block" made only of a deleted element.

Complexity. O(n) time, O(1) memory; one pass, so it also works on a stream.

How to test it live. All-negative input ($[-3, -1, -2] \to -1$), a single element, and a brute force over every block on small random arrays.

P8. Running median of a stream

Reported: *Quantt HRT guide, "Question 1: Stream median" among reported engineering questions; related QuantVault items on streaming quantiles (seen 2024)*

The task.

– Implement add(x) and median(), both faster than O(n) per call, as numbers arrive one at a time.

The approach. Two heaps: a max-heap holding the smaller half and a min-heap holding the larger half, kept within one element of each other in size. The median is the top of the larger heap, or the average of both tops. add is O(log n); median is O(1).

```python
import heapq

class StreamMedian:
    def __init__(self):
        self.lo = []   # max-heap of the smaller half (store negatives)
```

```
        self.hi = []   # min-heap of the larger half

    def add(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))   # move the largest small to hi
        if len(self.hi) > len(self.lo):                     # keep len(lo) >= len(hi)
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def median(self):
        if len(self.lo) > len(self.hi):
            return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2
```

```
# test: 2,000 random inserts, median() equals statistics.median after every one
```

How the code works, step by step.

1. **lo** holds the smaller half as a max-heap (Python's heapq is a min-heap, so values are stored negated); **hi** holds the larger half as a min-heap.
2. **add** pushes x into lo, then moves lo's largest value into hi. This guarantees every value in lo is ≤ every value in hi.
3. If hi now has more elements than lo, move hi's smallest back. The invariant afterwards: len(lo) equals len(hi) or exceeds it by one.
4. **median**: with an odd count the middle value is the top of lo; with an even count it is the average of the two tops.

Complexity. add is O(log n) (three heap operations); median is O(1).

How to test it live. After every add on a random stream, compare with statistics.median of everything so far.

P9. A set with O(1) insert, delete and random element — and a weighted version

Reported: *Glassdoor: Software Engineering, Jan 2021 ("design a data structure that adds and randomizes remove in O(1)"); Algorithm Developer, Dec 2020 ("push() and randompop() ... then weighted_randompop()"); Senior Network Automation Engineer, Aug 2022*

> **ALSO IN OA PREP** Same problem as **J3** in your HRT WiTTI CodeSignal OA Prep document. This version adds a step-by-step explanation of the Fenwick-tree sampler.

The task.

- (a) Support insert(x), remove(x) and get_random() (each element equally likely), all O(1) on average.
- (b) Follow-up: push(x, w) and pop an element at random with probability proportional to its weight.

The approach. (a) A list holds the values and a dictionary maps each value to its index. To remove, move the last element into the hole and pop the end, so nothing shifts. (b) Equal-probability tricks no longer work. Keep the weights in a Fenwick (binary indexed) tree of prefix sums; draw a uniform number in [0, total) and walk down the tree to find whose interval it lands in. Both push and pop are O(log n).

```
import random, bisect

class RandomizedSet:
    """insert, remove, get_random all O(1) on average."""
    def __init__(self):
        self.items = []          # the values, in any order
        self.pos = {}            # value -> index in items

    def insert(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
```

```
        self.items.append(x)
        return True

    def remove(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        self.items[i], self.pos[last] = last, i   # move last element into the hole
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        return random.choice(self.items)

class WeightedSampler:
    """push(x, w) and pop a random element with probability w / total.
    Fenwick tree over insertion slots: O(log n) per operation."""
    def __init__(self, capacity):
        self.n = capacity
        self.tree = [0.0] * (capacity + 1)
        self.vals = [None] * (capacity + 1)
        self.w = [0.0] * (capacity + 1)
        self.next = 1

    def _add(self, i, d):
        while i <= self.n:
            self.tree[i] += d
            i += i & -i

    def push(self, x, w):
        i = self.next; self.next += 1
        self.vals[i], self.w[i] = x, w
        self._add(i, w)

    def pop_random(self):
        target = random.random() * self.tree_total()
        i, step = 0, 1 << self.n.bit_length()
        while step:                       # descend the Fenwick tree
            j = i + step
            if j <= self.n and self.tree[j] <= target:
                i = j
                target -= self.tree[j]
            step >>= 1
        i += 1
        self._add(i, -self.w[i]); self.w[i] = 0.0
        return self.vals[i]

    def tree_total(self):
        s, i = 0.0, self.n
        while i > 0:
            s += self.tree[i]; i -= i & -i
        return s
```

```
# weights 1, 2, 3 -> empirical frequencies {a: 0.170, b: 0.331, c: 0.499}; expected 1/6, 1/3, 1/2
```

> **FOLLOW-UP** If the weights never change, Walker's alias method gives O(1) sampling after O(n) set-up. Mention it; the Fenwick tree handles the harder case where elements come and go.

How the code works, step by step.

1. (a) **items** is a list of the values; **pos** maps each value to its index in the list. insert appends and records the index.
2. **remove** copies the last element into the removed element's slot, updates that element's index in pos, then pops the end of the list. No other element moves, so it is O(1).

3. **get_random** is random.choice(items): every element is equally likely because the list has no gaps.
4. (b) A **Fenwick tree** stores partial sums: tree[i] covers a block of weights ending at slot i whose length is i's lowest set bit. **_add** updates the O(log n) blocks containing a slot.
5. **pop_random** draws a target in [0, total) and walks down from the largest power of two: if a whole block's weight is ≤ the remaining target, skip it and subtract. It stops at the slot whose weight interval contains the target, which happens with probability weight/total. That slot's weight is then set to 0.

Complexity. (a) O(1) average per operation. (b) O(log n) per push and pop.

How to test it live. Pop from weights 1, 2, 3 many times and check the frequencies come out near 1/6, 1/3, 1/2.

P10. Second-largest element with the fewest comparisons

Reported: *Glassdoor: Algorithm Developer, Dec 2020 ("there is an n + O(log n) answer"); Quant and Algo Developer final round, Oct 2022; QuantVault, "Second Largest with Minimum Comparisons", seen 2022*

The task.

– Find the second-largest of n distinct numbers using as few comparisons as possible. How many do you need?

The approach. Run a knockout tournament: $n - 1$ comparisons find the maximum. The second-largest lost only to the maximum, so it is among the $\lceil \log_2 n \rceil$ elements the maximum beat. A second mini-tournament among those costs $\lceil \log_2 n \rceil - 1$ more. Total: $n + \lceil \log_2 n \rceil - 2$, which is optimal.

```python
def second_largest(a):
    """Tournament: n-1 comparisons find the max; the runner-up must have lost
    directly to the max, and the max played only ceil(log2 n) matches."""
    comps = 0
    players = [(x, []) for x in a]           # (value, list of values it beat)
    while len(players) > 1:
        nxt = []
        for i in range(0, len(players) - 1, 2):
            (x, bx), (y, by) = players[i], players[i + 1]
            comps += 1
            nxt.append((x, bx + [y]) if x >= y else (y, by + [x]))
        if len(players) % 2:
            nxt.append(players[-1])
        players = nxt
    beaten = players[0][1]
    second = beaten[0]
    for v in beaten[1:]:
        comps += 1
        second = max(second, v)
    return second, comps
```

```python
# n = 8: 9 comparisons;  n = 100: 105;  n = 1000: 1008  (each equals n + ceil(log2 n) - 2)
```

> **TRAP** The naive "track the top two in one pass" uses up to $2n - 3$ comparisons. The interviewer wants the tournament argument and the count, not just correct code.

How the code works, step by step.

1. Each player is (value, list of values it has beaten).
2. Each round pairs neighbours; the larger value advances and records the loser. An odd player out advances without a comparison.
3. After the last round the champion is the maximum. It played one match per round — about $\log_2 n$ matches — so its beaten list is short.
4. The second-largest must have lost to the maximum (nothing else could beat it), so it is the largest value in that list: one more mini-search.

Complexity. $n - 1$ comparisons for the tournament plus $\lceil \log_2 n \rceil - 1$ for the beaten list: $n + \lceil \log_2 n \rceil - 2$ in total.

How to test it live. Random distinct lists of many sizes; assert the answer equals sorted(a)[−2] and print the comparison count next to the formula.

P11. The polydivisible pandigital number (a digit-divisibility search)

Reported: *Glassdoor: Software Engineer, Mar 2024 ("Find the polydivisible pandigital number"); Sophomore Program final interview, Feb 2024 — one live coding problem: "how many numbers can be divided by (0–9) with only 0–9 number in each digits?", most plausibly this problem*

The task.

- Find the 10-digit number that uses each digit 0–9 exactly once and whose first k digits form a number divisible by k, for every k from 1 to 10.
- Follow-up: count all polydivisible numbers (digits may repeat, no leading zero).

The approach. Backtracking: build the number one digit at a time, and abandon a prefix the moment its first k digits are not divisible by k. The pruning is so strong that the search is instant. The follow-up uses the same idea breadth-first: extend every survivor of length $k - 1$ by each digit and keep those divisible by k, until none survive.

```python
def polydivisible_pandigital():
    """10-digit numbers using each digit 0-9 once, whose first k digits are
    divisible by k for every k = 1..10."""
    out = []
    def extend(prefix, value, used):
        k = len(prefix)
        if k == 10:
            out.append(value); return
        for d in range(10):
            if used >> d & 1 or (k == 0 and d == 0):
                continue
            v = value * 10 + d
            if v % (k + 1) == 0:              # prune as soon as a prefix fails
                extend(prefix + [d], v, used | 1 << d)
    extend([], 0, 0)
    return out

def count_polydivisible(max_len):
    """How many polydivisible numbers (digits may repeat) have <= max_len digits."""
    level = list(range(1, 10)); total = len(level); k = 1
    while level and k < max_len:
        k += 1
        level = [v * 10 + d for v in level for d in range(10) if (v * 10 + d) % k == 0]
        total += len(level)
    return total, k
```

```python
# polydivisible_pandigital() -> [3816547290]   (the only one)
# count_polydivisible(30)     -> (20456, 26): 20,456 polydivisible numbers; none has more than 25 digits
```

> **SAY THIS** Before coding, mention the hand-reasoning that makes it feel inevitable: the 10th digit must be 0 and the 5th must be 5; even positions take the even digits. The program then confirms there is exactly one answer.

How the code works, step by step.

1. **extend(prefix, value, used)** builds the number digit by digit. **used** is a bitmask: bit d is set once digit d has been placed.
2. Skip a digit that is already used, and skip 0 as the first digit.
3. Appending d gives $v = 10 \cdot \text{value} + d$ with $k + 1$ digits; keep it only if v is divisible by $k + 1$. A failing prefix is abandoned immediately, which prunes almost the whole search tree.

4. At 10 digits every condition has been checked, so the number is recorded.
5. **count_polydivisible** works level by level: all valid 1-digit numbers, then every valid number extended by one digit and filtered by divisibility, until no number survives.

Complexity. The pandigital search visits only a few thousand prefixes. The counting version holds at most a few thousand numbers per level and stops at 25 digits.

How to test it live. Check that the answer 3816547290 really works: 3 % 1, 38 % 2, 381 % 3, ... 3816547290 % 10 all equal 0.

P12. Flip intervals of a bit array

Reported: *LinkJob 2025 HRT write-up: "Flipping an array of size N with all elements as 0. Given some intervals [a, b], flipping all the state within the interval. How to determine the final status of the array?"*

The task.

- An array of n zeros. Each interval [a, b] (inclusive) flips every bit in it.
- Return the final array, faster than flipping element by element.

The approach. Difference array: toggle a marker at a and at b + 1. The running XOR of the markers equals the number of flips covering each position, mod 2. O(n + m) for m intervals instead of O(n·m).

```python
def final_state(n, intervals):
    """Array of n zeros; each [a, b] (inclusive, 0-based) flips every bit in it.
    Difference array: mark +1 at a and at b+1; prefix parity = flip count mod 2."""
    diff = [0] * (n + 1)
    for a, b in intervals:
        diff[a] ^= 1
        diff[b + 1] ^= 1
    out, cur = [], 0
    for i in range(n):
        cur ^= diff[i]
        out.append(cur)
    return out
```

```python
# final_state(8, [(1, 4), (3, 6)]) -> [0, 1, 1, 0, 0, 1, 1, 0]; checked against brute force
```

How the code works, step by step.

1. **diff[a] ^= 1** marks where an interval starts flipping; **diff[b + 1] ^= 1** marks where it stops. XOR is used instead of +1/−1 because only the parity matters.
2. Scanning left to right, **cur ^= diff[i]** is the parity of (intervals started − intervals ended) up to i — exactly the number of intervals covering i, mod 2.
3. An even number of flips leaves a 0; an odd number gives 1, so cur is the final bit.

Complexity. O(n + m) for m intervals, versus O(n·m) for flipping element by element.

How to test it live. Compare with the slow element-by-element version on random small cases (as in the test).

P13. Count rectangles and squares whose corners are given points

Reported: *Glassdoor: Python Algorithm Developer intern phone screen, Feb 2024 ("count the number of distinct rectangles that can be drawn by connecting any 4 points"); Summer intern OA, Mar 2024; Algorithm Development intern OA, Oct 2022 (non-axis-aligned squares and rectangles)*

The task.

- Given n distinct integer points, count the rectangles (any orientation) whose four corners are all in the set.
- Follow-up: count the squares.

The approach. Rectangles: two point pairs are the diagonals of a rectangle exactly when they share a midpoint and have equal length. Group all pairs by (sum of coordinates, squared length) — integers, so no floating point — and any two pairs in the same group form one rectangle: add $m(m-1)/2$ per group. $O(n^2)$. Squares: treat each pair as one side, rotate it 90° both ways and check whether the other two corners exist; each square is found once per side, so divide by 4.

```python
from collections import defaultdict
from itertools import combinations

def count_rectangles(points):
    """Rectangles (any orientation) with all 4 corners in `points` (distinct).
    Two segments are the diagonals of a rectangle iff they share a midpoint and
    have equal length. Group point pairs by (midpoint*2, squared length)."""
    groups = defaultdict(int)
    for (x1, y1), (x2, y2) in combinations(points, 2):
        key = (x1 + x2, y1 + y2, (x1 - x2) ** 2 + (y1 - y2) ** 2)
        groups[key] += 1
    return sum(m * (m - 1) // 2 for m in groups.values())

def count_squares(points):
    """Squares (any orientation): for each pair as one side, check the other two corners."""
    s = set(points); cnt = 0
    for (x1, y1), (x2, y2) in combinations(points, 2):
        dx, dy = x2 - x1, y2 - y1
        if (x1 - dy, y1 + dx) in s and (x2 - dy, y2 + dx) in s:
            cnt += 1
        if (x1 + dy, y1 - dx) in s and (x2 + dy, y2 - dx) in s:
            cnt += 1
    return cnt // 4          # each square is found once from each of its 4 sides
```

```python
# 3 x 3 lattice: 10 rectangles (9 axis-aligned + 1 tilted), 6 squares (4 unit, 1 large, 1 tilted)
```

How the code works, step by step.

1. For every pair of points, compute **$(x_1 + x_2, y_1 + y_2,$ squared length)**. Using the doubled midpoint keeps everything integer, so no floating-point comparisons.
2. Two segments are the diagonals of a rectangle exactly when they bisect each other (same midpoint) and have equal length. So any two pairs with the same key form one rectangle.
3. A group of m pairs with the same key gives $m(m-1)/2$ rectangles.
4. For squares: treat a pair as one side with vector (dx, dy). Rotating it 90° gives $(-dy, dx)$; the square's other two corners are the pair shifted by that vector (or by its negative). Check both in a set.
5. Every square is found once from each of its 4 sides, so divide the count by 4.

Complexity. $O(n^2)$ pairs; $O(n^2)$ memory for the groups.

How to test it live. The 3 × 3 lattice: 9 axis-aligned rectangles plus 1 tilted square = 10; 4 unit + 1 large + 1 tilted = 6 squares.

P14. Convex hull of a set of points

Reported: *Glassdoor, Intern coding interview, Dec 2025 ("Find all the points that make up the convex hull of a set of points"); QuantVault, "Convex Hull via Monotone Chain", seen 2024*

The task.

– Given points in the plane, return the vertices of their convex hull in order.

The approach. Andrew's monotone chain: sort the points, then build the lower and upper hulls with a stack, popping the last point whenever it does not make a left turn (cross product ≤ 0). $O(n \log n)$ for the sort, $O(n)$ for the scan.

```python
def convex_hull(points):
    """Andrew's monotone chain, O(n log n). Returns hull counter-clockwise,
    no collinear points, starting from the lowest-x point."""
    pts = sorted(set(points))
    if len(pts) <= 2:
        return pts
    def cross(o, a, b):
        return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
    lower, upper = [], []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]
```

```python
# convex_hull([(0,0),(2,0),(1,1),(2,2),(0,2),(1,0),(1,3)]) -> [(0,0), (2,0), (2,2), (1,3), (0,2)]
```

> **TRAP** Collinear points and duplicates: say whether points on an edge belong on the hull (here they do not, because of ≤ 0), and de-duplicate first.

How the code works, step by step.

1. Sort the points by x, then y, after removing duplicates.
2. **cross(o, a, b)** is the z-component of $(a - o) \times (b - o)$: positive means o → a → b turns left.
3. Lower hull: go left to right. Before adding p, pop the last point while the last two points and p do not make a strict left turn — that point lies inside or on the hull.
4. Upper hull: the same scan from right to left.
5. Join them, dropping each list's last point because it repeats the start of the other list.

Complexity. $O(n \log n)$ for the sort; the scans are $O(n)$ because each point is pushed and popped at most once.

How to test it live. A square with an interior point and points on the edges: the output should be exactly the four corners, counter-clockwise.

P15. Fancy numbers: count base-4 numbers made only of 0s and 1s

Reported: *Glassdoor: Algorithm Developer intern OA, Apr 2025 ("A pretty number is a number in base 4 whose digits are all either 1 or 0. How many pretty numbers are there < n?"); Quant Research intern OA, Nov 2025; Algorithm Developer OA, Mar 2025 and Quantitative Developer, Mar 2025 ("design an algo to find the fancy number"). Quantt and Tradermath both list it as the most-reported HRT OA problem (n up to 10^9, C++ limit 0.5 s)*

The task.

- A positive integer is "fancy" if its base-4 representation uses only the digits 0 and 1 (examples: $1, 4 = 10_4$, $5 = 11_4$, $16 = 100_4$).
- Given n (up to 10^9), return how many fancy numbers are strictly less than n.
- Checking every number below n is too slow at this size.

The key observation. Write a fancy number's base-4 digits and read them as a binary number: $1 \leftrightarrow 1$, $4 = 10_4 \leftrightarrow 10_2 = 2$, $5 = 11_4 \leftrightarrow 3$, $16 = 100_4 \leftrightarrow 4$. This is a one-to-one map between fancy numbers and positive integers, and **it keeps order**: comparing two 0/1 digit strings works the same way in base 2 and base 4. So the k-th fancy number is simply k written in binary and read in base 4.

Solution 1 — enumerate in order. Generate the fancy numbers for k = 1, 2, 3, ... and stop at the first one that is $\geq$ n. Since $4^{15} \approx 1.07 \times 10^9$, a fancy number below 10^9 has at most 15 digits, so there are at most $2^{15} - 1 = 32{,}767$ of them — trivially fast.

Solution 2 — count digit by digit, O(log n). Walk n's base-4 digits from the most significant. A digit 0 forces a 0 in the answer; a digit 1 lets you place 0 here (then every lower position is free: 2^rest options) or place 1 and keep matching n; a digit 2 or 3 lets you place 0 or 1 here with every lower position free ($2 \cdot 2^{rest}$), and then you must stop, because no fancy number can match a 2 or 3. The walk counts 0 too, so subtract 1.

```python
def count_fancy_enumerate(n):
    """Positive integers below n whose base-4 digits are all 0 or 1.
    The k-th fancy number is k written in binary, then read in base 4;
    that map keeps order, so list them in increasing order and stop at n."""
    count, k = 0, 1
    while True:
        value, bits, place = 0, k, 1
        while bits:                       # read the binary digits of k in base 4
            value += (bits & 1) * place
            bits >>= 1
            place *= 4
        if value >= n:
            return count
        count += 1
        k += 1

def count_fancy_digits(n):
    """Same count in O(log n): walk n's base-4 digits from the top."""
    if n <= 1:
        return 0
    digits = []
    while n:
        digits.append(n % 4)
        n //= 4
    digits.reverse()                      # most significant first
    count = 0
    for i, d in enumerate(digits):
        rest = len(digits) - i - 1        # positions still free below this one
        if d == 0:
            continue                      # must place 0 here to stay below n
        if d == 1:
            count += 2 ** rest            # place 0 here: any 0/1 digits below
            continue                      # or place 1 and keep matching n
        count += 2 * 2 ** rest            # d >= 2: place 0 or 1, rest free
        break                             # cannot place d itself (not 0/1)
    return count - 1                      # the walk counted 0; drop it
```

```python
# n = 5     -> 2          (1 and 4; 5 = 11 in base 4 is not below 5)
# n = 10**9  -> 32767       (both functions)
# n = 10**18 -> 1073741823  (count_fancy_digits only)
# both checked against brute force for every n < 3000 and 300 random n up to 100,000
```

SAY THIS Explain why the order is kept before writing any code — that is what the interviewer wants to hear. Then mention the O(log n) digit walk as the follow-up for much larger n.

TRAP Strictly less than n: n itself is excluded (n = 5 gives 2, not 3). And handle $n \leq 1$, where the answer is 0.

How the code works, step by step.

1. **count_fancy_enumerate**: for k = 1, 2, 3, ..., read k's binary digits (bits & 1, then bits »= 1) as base-4 digits (place *= 4). Stop at the first value ≥ n; the count so far is the answer.
2. **count_fancy_digits** first lists n's base-4 digits, most significant first.
3. At each position, **rest** is how many lower positions remain. A 0 forces 0. A 1 allows placing 0 here (then 2^rest free choices below) or matching with 1 and continuing.
4. A 2 or 3 allows 0 or 1 here with everything below free ($2 \cdot 2^{rest}$), then stops: no fancy number can match a 2 or 3.

5. The walk also counted 0, which is not positive, so subtract 1.

Complexity. Enumeration: O(answer × log n), at most 32,767 steps for $n \le 10^9$. Digit walk: O(log n).

How to test it live. Brute force (check every number below n) for $n < 3000$ must agree with both functions.

More reported live-coding problems (P16–P27), mostly from 2026 technical screens

These come from candidate write-ups collected on PracHub (each linked in Part 6) and from Glassdoor. PracHub rewrites the statements in full, so the exact rules below are its versions; the candidate reports it links to are first-hand. Several are from 30–60-minute HRT screens with one coding problem — the same time budget as your programming half.

> **KEY** The 30-minute reports share one lesson. A candidate who wrote a bug-free, optimised solution was still rejected because he had assumed one rule wrongly (whether certain grid cells could be walked on) and had no time to change the code. **Spend the first two minutes confirming every rule with the interviewer, and say your assumptions out loud.**

P16. Mean or median? Estimate μ when 20% of the data sit exactly on μ — then simulate

Reported: *PracHub, candidate report: HRT Data Scientist first Zoom interview, Sep 2026 — "the interviewer asked me to write a simulation". The closest match to the "derive, then simulate" format, and very recent*

The task.

- Each observation X equals μ exactly with probability 0.2, and is drawn from $N(\mu, 1)$ with probability 0.8.
- Q1: given n samples, how would you estimate μ? Sample mean or sample median?
- Q2: write a simulation comparing their mean absolute errors (MAE).
- Q3: for $n = 80$, estimate $P(|\text{estimate} - \mu| > 0.1)$ for each.

The answer. The median is far better. For n = 80: MAE 0.080 for the mean against 0.005 for the median; P(error > 0.1) is 0.317 for the mean against about 0.015 for the median.

The reasoning first (say this before coding). Both estimators are unbiased because the distribution is symmetric about μ. The mean averages everything: its variance is $\mathrm{Var}(X)/n = 0.8/n$, so for $n = 80$ its standard deviation is exactly 0.1, and $P(|\text{error}| > 0.1) = P(|Z| > 1) \approx 0.317$. The median, by contrast, lands **exactly** on μ whenever the 20% point mass straddles the middle of the sorted sample — about 16 of the 80 observations sit precisely at μ, so the middle of the sorted data very often is μ itself.

```python
import numpy as np

def sample(n, mu, rng, reps):
    """reps x n draws: with prob 0.2 exactly mu, else N(mu, 1)."""
    x = rng.normal(mu, 1.0, size=(reps, n))
    point = rng.random((reps, n)) < 0.2
    x[point] = mu
    return x

def compare(n=80, mu=0.0, reps=100_000, seed=0):
    rng = np.random.default_rng(seed)
    x = sample(n, mu, rng, reps)
    err_mean = np.abs(x.mean(axis=1) - mu)
    err_med = np.abs(np.median(x, axis=1) - mu)
    print(f"MAE   mean={err_mean.mean():.4f}  median={err_med.mean():.4f}")
    print(f"P(|err|>0.1) mean={np.mean(err_mean > 0.1):.4f}  median={np.mean(err_med > 0.1):.4f}")
```

```
# output (n = 80, 100,000 simulated data sets):
# MAE   mean=0.0796  median=0.0047
# P(|err|>0.1) mean=0.3168  median=0.0151
```

How the code works, step by step.

1. **sample** draws a reps × n table of $N(\mu, 1)$ values, then overwrites a random 20% of the entries with μ itself (a boolean mask from rng.random(...) < 0.2). One table = 100,000 independent data sets of size 80.
2. **x.mean(axis=1)** and **np.median(x, axis=1)** give one estimate per data set; subtracting μ and taking absolute values gives the errors.
3. The **MAE** is the average error; **np.mean(err > 0.1)** is the fraction of data sets whose error exceeds 0.1 — the probability asked for in Q3.
4. Compare the mean's numbers with theory: standard deviation $\sqrt{(0.8/80)} = 0.1$, so MAE $= 0.1 \cdot \sqrt{(2/\pi)} \approx 0.080$ and $P(|Z| > 1) \approx 0.317$. Matching theory shows the simulation is right before you trust the median's numbers.

Complexity. O(reps × n log n) for the medians (a sort per row), all inside numpy.

How to test it live. Set the point-mass probability to 0: the median should then be worse than the mean (its variance is about $\pi/2$ times larger for normal data). That reversal is a good follow-up to mention.

P17. Root-to-leaf path sum in a binary tree, then all such paths

Reported: *PracHub, candidate report: HRT C++ Software Engineer **intern**, first round, Jul 2026 — a 1-hour Zoom coding interview, "easier than I expected … felt like a LeetCode Easy"; the interviewer cared most about edge cases and explanation*

> **ALSO IN OA PREP** Same core problem as **F1** in your OA prep document. New here: the "return every path" follow-up, the July 2026 intern-round context, and the step-by-step explanation.

The task.

- Given a binary tree of integers (values may be negative) and a target, is there a path from the root down to a **leaf** whose values sum to the target?
- An empty tree returns False. The path must end at a leaf — it cannot stop at an internal node.
- Follow-up: return every such path.

The approach. Depth-first search. At each node subtract its value from the target and pass what is left to the children. At a leaf, the path works exactly when the remaining target equals the leaf's value. For the follow-up, keep the current path in a list: append on the way down, record a copy at a valid leaf, and pop on the way back up (backtracking).

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def has_path_sum(root, target):
    """True if some root-to-leaf path sums to target. Empty tree -> False."""
    if root is None:
        return False
    if root.left is None and root.right is None:      # a leaf: the path ends here
        return root.val == target
    rest = target - root.val                          # what the children still need
    return has_path_sum(root.left, rest) or has_path_sum(root.right, rest)

def all_path_sums(root, target):
    """Every root-to-leaf path (as a list of values) whose sum is target."""
    out, path = [], []
    def dfs(node, remaining):
        if node is None:
            return
        path.append(node.val)                         # choose
        remaining -= node.val
        if node.left is None and node.right is None:
            if remaining == 0:
                out.append(path.copy())               # copy: path keeps changing
        else:
            dfs(node.left, remaining)
            dfs(node.right, remaining)
        path.pop()                                    # un-choose (backtrack)
```

```
    dfs(root, target)
    return out
```

```
# all_path_sums(tree, 22) -> [[5, 4, 11, 2], [5, 8, 4, 5]]
# tree 1 -> 2 with target 1 -> False: node 1 has a child, so it is not a leaf
```

How the code works, step by step.

1. **has_path_sum**: an empty subtree can never complete a path, so it returns False. That is also why a node with only one child must not count its missing side as a leaf.
2. A node with no children is a **leaf**: the path ends here and works exactly when root.val == target (what is still needed).
3. Otherwise, the children need **target − root.val** more; the answer is True if either side can supply it. Python's **or** stops early once one side succeeds.
4. **all_path_sums** keeps one shared list **path**. Append the node (choose), recurse, then **pop** (un-choose) so the list is correct for the next branch.
5. At a valid leaf, store **path.copy()** — storing path itself would store a list that keeps changing and ends up empty.

Complexity. O(n) time for both; O(h) extra memory for recursion depth h (O(n) for a fully skewed tree). The follow-up also spends O(h) per path it copies.

How to test it live. Empty tree; a single node equal and not equal to the target; negative values (−2 → −3 with target −5); a node with one child (must not stop there); the classic tree above with two valid paths.

> **FOLLOW-UP** Iterative version: a stack of (node, remaining) pairs avoids recursion limits on very deep trees. Mention it; write the recursive one.

P18. Where is the ball after the maze is tilted at given times?

Reported: *PracHub, candidate report: HRT new-grad Software Engineer, one 30-minute coding round, Jul 2026 (rejected after a perfect OA). A related version — a moving object on a grid with timestamped direction changes and "dot" obstacles — was reported in Aug 2026 by the candidate who misread whether dots were walkable*

The task.

- A rectangular maze of "#" (wall) and "." (open) holds one ball at start = [row, col].
- The board is "flat" until the first event. Each event (t, state) sets the state to "up", "down", "left", "right" or "flat" from second t onward.
- In every second s = 0, 1, ..., end_time − 1, if the board is tilted and the next cell in that direction is open and inside the maze, the ball moves into it at the end of the second.
- Times go up to 10^9. Return the ball's cell at end_time.

The approach. Simulating second by second is too slow for 10^9. But the state only changes at event times, so process one **segment** $[t_i, t_{i+1})$ at a time. Within a segment the ball rolls until blocked — and once blocked it stays blocked until the state changes — so each segment costs at most the maze's length, however long it lasts.

```
DIRS = {"up": (-1, 0), "down": (1, 0), "left": (0, -1), "right": (0, 1)}

def ball_position(maze, start, events, end_time):
    R, C = len(maze), len(maze[0])
    r, c = start
    # board is flat until the first event; turn events into segments [t, next_t)
    times = [t for t, _ in events if t < end_time]
    states = [s for t, s in events if t < end_time]
    for i, (t, state) in enumerate(zip(times, states)):
        seg_end = times[i + 1] if i + 1 < len(times) else end_time
```

```
        if state == "flat":
            continue
        dr, dc = DIRS[state]
        steps = seg_end - t                      # seconds in this segment
        while steps > 0:                         # stop early once blocked
            nr, nc = r + dr, c + dc
            if not (0 <= nr < R and 0 <= nc < C) or maze[nr][nc] == "#":
                break                            # blocked for the rest of the segment
            r, c = nr, nc
            steps -= 1
    return [r, c]
```

```
# example 1 -> [1, 5];  example 2 -> [1, 1];  2 x 2 example -> [1, 1]
# 3,000 random mazes: segment version agrees with the second-by-second brute force
```

How the code works, step by step.

1. **DIRS** maps each tilt to a (row, column) step. Row 0 is the top, so "up" is −1 in the row.
2. Events at or after end_time can never act, so they are dropped first; this also makes end_time = 0 return start.
3. For event i, its segment ends at the next event's time or at end_time — **seg_end − t** seconds of the same state.
4. A "flat" segment changes nothing. For a tilt, move one cell at a time while seconds remain and the next cell is inside the maze and not a wall.
5. **break** at the first blocked step: the maze and the tilt do not change during the segment, so the ball would stay blocked for all remaining seconds.

Complexity. O(E × max(R, C)) for E events: each segment moves at most the maze's width or height. O(1) extra memory.

How to test it live. The three given examples, an end_time of 0, an event exactly at end_time (ignored), a 1 × 1 maze, then compare with a second-by-second brute force on small random mazes with small times.

> **TRAP** Off-by-one timing is the whole difficulty: an event at time t governs second t; moves happen at the end of a second; only seconds 0 … end_time − 1 count. Restate these rules to the interviewer before coding.

P19. Players, a watcher who turns around, and a finish line — plus itoa

Reported: *PracHub, candidate reports: HRT Software Engineer technical screens, Nov 2025 and 2026 ("a Watcher-and-Players Puzzle"); the integer-to-string task was the second question of the same screen*

> **ALSO IN OA PREP** The watcher problem is the same as **H2** in your OA prep document. New here: the integer-to-string (itoa) task from the same screen, and the step-by-step explanation.

Task 1.

- A track runs from 0 to L (the finish). Player i starts at players[i]; every player wants to move right one unit per second.
- A watcher stands at w, initially looking right. At each time in turn_times (unsorted, may repeat) the watcher turns around.
- In second t: apply turns at t, then a player is **watched** if the watcher looks right and the player is at ≥ w, or looks left and the player is at ≤ w. Watched players freeze; the rest move one step. Players at L are finished.
- How many players have finished by time T (T up to 10^9)?

Task 2.

- Convert a 32-bit signed integer to its decimal string without using str() or similar, including −2147483648.

The key insight. Players never interact and only move right, so each player's fate depends only on where it starts relative to w. A player right of w can move only while the watcher looks left; a player left of w can move only while it looks right, and only up to w — where it is watched in both directions and freezes for ever (unless w is the finish). So all you need is **how many seconds the watcher spends facing each way**.

```
def facing_seconds(turn_times, T):
    """Seconds in [0, T) the watcher faces right and left. Starts facing right;
    each turn at time t (applied before second t) flips the direction."""
    right = left = 0
    facing_right, prev = True, 0
    for t in sorted(x for x in turn_times if x < T):
        if facing_right: right += t - prev
        else:            left += t - prev
        facing_right, prev = not facing_right, t
    if facing_right: right += T - prev
    else:            left += T - prev
    return right, left

def count_finished(L, players, w, turn_times, T):
    right, left = facing_seconds(turn_times, T)
    done = 0
    for p in players:
        if p == L:
            done += 1                       # already finished
        elif p > w:
            done += left >= L - p           # free only while the watcher looks left
        elif p < w and w == L:
            done += right >= L - p          # free only while it looks right; w is the finish
        # p < w < L: it can only walk up to w, where it is always watched
        # p == w < L: always watched, never moves
    return done
```

```
# examples -> 2 and 3; 4,000 random cases agree with a second-by-second brute force
# int_to_decimal_string agrees with str() on 0, ±7, ±10, 2**31-1, -2**31 and 2,000 random values
```

How the code works, step by step.

1. **facing_seconds** sorts the turns before T and walks through them, adding each gap to whichever direction the watcher was facing; the last stretch runs to T. Two turns at the same time cancel automatically (a gap of 0 seconds).
2. In **count_finished**, a player already at L counts immediately.
3. A player right of w needs $L - p$ free seconds and is free only while the watcher looks left: it finishes if **left** $\geq$ **L − p**. (The order of the seconds does not matter — only how many.)
4. A player left of w can only advance while the watcher looks right, and stops for good at w. It can finish only if **w == L** and **right** $\geq$ **L − p**.
5. A player starting exactly at w (below L) is watched in both directions and never moves.
6. **int_to_decimal_string**: handle 0 separately; remember the sign; repeatedly take **n % 10** (last digit) and **n //= 10** (drop it), building the digits backwards; add "-" and reverse.

Complexity. Task 1: O(K log K + N) for K turns and N players — no dependence on T. Task 2: O(number of digits).

How to test it live. Task 1: the two examples, then a brute-force simulation on small random cases (as in the test). Task 2: 0, a negative, 10 (a trailing zero) and the extremes $\pm 2^{31}$.

> **TRAP** In C++, negating −2147483648 overflows a 32-bit int. Use a 64-bit integer for the magnitude, or build the digits from the negative side. Python has no overflow, but say this out loud — it is exactly the detail the question tests.

P20. The point lit by the most street lamps

Reported: *PracHub, from the same HRT new-grad report as P18 (Jul 2026), listed as an OA problem*

The task.

- Lamp i stands at c with radius r and lights every point in $[c - r, c + r]$ (both ends included).
- Return the integer point lit by the most lamps; on a tie, the smallest such point. Up to 10^5 lamps; coordinates up to $\pm 2 \cdot 10^9$.

The approach. A sweep line. The coverage count only changes where an interval starts or just after one ends, so record $+1$ at $c - r$ and -1 at $c + r + 1$ (one past the inclusive end), sort, and scan. The best point is always the left end of some interval.

```python
def most_lit_point(lamps):
    """Smallest integer point covered by the most closed intervals [c-r, c+r]."""
    events = []
    for c, r in lamps:
        events.append((c - r, +1))          # coverage starts at c - r
        events.append((c + r + 1, -1))      # and stops just after c + r (inclusive end)
    events.sort()                           # at equal x, -1 sorts before +1 (harmless: all applied before reading)
    best_count, best_x, count, i = 0, None, 0, 0
    while i < len(events):
        x = events[i][0]
        while i < len(events) and events[i][0] == x:   # apply every event at x first
            count += events[i][1]
            i += 1
        if count > best_count:              # strictly greater keeps the smallest x on ties
            best_count, best_x = count, x
    return best_x
```

```python
# [[-2,3],[1,2],[5,1]] -> -1;  [[0,1],[2,1],[6,2],[7,1]] -> 1;  [[10,0],[3,2],[20,5]] -> 1
```

How the code works, step by step.

1. Each lamp produces two events: **$(c - r, +1)$** where its light starts and **$(c + r + 1, -1)$** just after it ends. Putting the end one past $c + r$ keeps the right end lit, so two lamps that touch at a point both count there.
2. Sort the events by position.
3. Apply **every** event at the same position before reading the count — otherwise a lamp ending and another starting at the same x would be double-counted or missed.
4. Update the answer only when the count is **strictly** larger. Because positions are scanned left to right, the first position that reaches the maximum — the smallest — is kept.

Complexity. $O(n \log n)$ for the sort; $O(n)$ memory.

How to test it live. The three examples; touching ranges; a radius-0 lamp; then a brute force over every integer point on small random inputs.

P21. Largest filled diamond in a grid

Reported: *PracHub, from the same HRT new-grad report as P18 (Jul 2026)*

The task.

- A diamond with centre (r, c) and radius k is every cell with $|i - r| + |j - c| \le k$. It must fit entirely in the grid; values can be negative.
- Return the largest diamond sum (radius 0, a single cell, is allowed).

The approach. A diamond is a stack of horizontal segments: row $r + d$ has half-width $k - |d|$. Precompute **row prefix sums**, so each segment is one subtraction. Try every centre and every radius that fits.

```python
def max_diamond_sum(grid):
    """Max sum over filled diamonds |i-r| + |j-c| <= k that fit in the grid.
    Row-prefix sums make each diamond O(k): a diamond is 2k+1 horizontal segments."""
    m, n = len(grid), len(grid[0])
    pre = [[0] * (n + 1) for _ in range(m)]          # pre[i][j] = sum of grid[i][0:j]
    for i in range(m):
        for j in range(n):
            pre[i][j + 1] = pre[i][j] + grid[i][j]
    best = max(max(row) for row in grid)             # radius 0: single cells
    for r in range(m):
        for c in range(n):
            kmax = min(r, c, m - 1 - r, n - 1 - c)
            for k in range(1, kmax + 1):             # every valid radius is a candidate
                total = 0
                for d in range(-k, k + 1):
                    w = k - abs(d)                   # half-width of row r+d
                    total += pre[r + d][c + w + 1] - pre[r + d][c - w]
                best = max(best, total)
    return best
```

```python
# [[1,2,3],[4,5,6],[7,8,9]] -> 25;  the negative example -> 10;  third example -> 11
```

How the code works, step by step.

1. **pre[i][j]** is the sum of the first j values of row i, so the sum of columns a ... b is pre[i][b + 1] − pre[i][a].
2. Start **best** at the largest single cell (all radius-0 diamonds).
3. For centre (r, c) the largest radius that fits is $\mathbf{min(r, c, m - 1 - r, n - 1 - c)}$.
4. For each radius k, rows r − k ... r + k contribute segments of half-width $\mathbf{k - |d|}$; add them with the prefix sums. Every radius is tried because, with negative values, the biggest diamond is not necessarily the best.

Complexity. $O(m \cdot n \cdot K^2)$ for maximum radius K — fine for small grids. The faster recurrence (diamond(k) built from two diamonds of radius k − 1 one row up and down) reaches $O(m \cdot n \cdot K)$; mention it as the optimisation.

How to test it live. The three examples, a 1 × 1 grid, an all-negative grid, then brute force on small random grids.

P22. A Wordle solver

Reported: *PracHub, HRT Software Engineer technical screen, Jul 2025*

The task.

– A hidden target word of length L comes from a known dictionary. A guess gets per-letter feedback: 2 = right letter, right place; 1 = in the word elsewhere; 0 = not in the word.

– Write solve(dictionary, target, max_attempts) that keeps guessing, prunes the candidates after each feedback, handles repeated letters correctly, and picks good guesses.

The approach. Get the scoring function exactly right first (two passes, for repeated letters). Then pruning needs no special bookkeeping: the target must be a word that would have produced **the same feedback** for the same guess, so keep exactly those words. Choose the next guess by letter frequency.

```python
from collections import Counter

def score(guess, target):
    """Wordle feedback: 2 = right letter right place, 1 = elsewhere, 0 = absent.
    Two passes so repeated letters are only credited as often as they occur."""
    fb = [0] * len(guess)
    unmatched = Counter()
    for i, (g, t) in enumerate(zip(guess, target)):   # pass 1: exact matches
        if g == t:
            fb[i] = 2
```

```
        else:
            unmatched[t] += 1                          # target letters still available
    for i, g in enumerate(guess):                      # pass 2: misplaced letters
        if fb[i] == 0 and unmatched[g] > 0:
            fb[i] = 1
            unmatched[g] -= 1
    return tuple(fb)

def choose(cands):
    """Pick the candidate whose distinct letters appear in the most candidates."""
    freq = Counter(ch for w in cands for ch in set(w))
    return min(cands, key=lambda w: (-sum(freq[ch] for ch in set(w)), w))

def solve(dictionary, target, max_attempts=6):
    if not dictionary:
        return "", 0
    cands = sorted(set(dictionary))
    guess = ""
    for attempt in range(1, max_attempts + 1):
        guess = choose(cands)
        fb = score(guess, target)
        if fb == (2,) * len(guess):
            return guess, attempt
        # keep only words that would have produced exactly this feedback
        cands = [w for w in cands if score(guess, w) == fb]
    return guess, max_attempts
```

```
# score("allee", "apple") -> (2, 1, 0, 0, 2): one L is present, the second L is absent
# solve(..., 'ghost') -> ('ghost', 2);  solve(..., 'apple') -> ('apple', 2)
```

How the code works, step by step.

1. **score**, pass 1: mark exact matches 2, and count the target letters that were not matched exactly (**unmatched**).
2. Pass 2: a guess letter that is not exact gets 1 only if an unmatched copy is still available, and uses it up. This is why a repeated guess letter can be 1 once and 0 the second time.
3. **choose** counts, for each letter, how many candidates contain it, and picks the candidate whose distinct letters cover the most candidates (ties broken alphabetically, so runs are repeatable).
4. **solve** guesses, stops on all 2s, otherwise filters the candidates to words w with **score(guess, w) == feedback**.

Complexity. Each round scores every candidate once: O(|candidates| × L). The candidate list shrinks quickly, usually by a large factor per guess.

How to test it live. Hand-check score on repeated-letter cases first, then run solve for every word in a small dictionary as the target and assert it is found.

P23. Evaluate an arithmetic expression (write a parser)

Reported: *Glassdoor: Algorithm Developer in-person onsite, Jan 2022 ("write a parser to evaluate math expressions"); Software Engineer intern OA, Nov 2022 ("complete the calculator class"); Senior Algo Engineer OA, Oct 2022 ("calculator")*

The task.

– Evaluate a string such as "2 * (3 + (4 − 1)) / 3" with + − * /, parentheses and unary minus, respecting precedence.

The approach. Recursive descent: one function per precedence level. expr handles + and −, term handles * and /, factor handles numbers, parentheses and unary minus. Because expr calls term and term calls factor, * and / automatically bind tighter than + and −.

```
def evaluate(expr):
    """Evaluate + - * / and parentheses with correct precedence (recursive descent).
    Grammar:  expr   := term   (('+' | '-') term)*
```

```
            term   := factor (('*' | '/') factor)*
            factor := number | '(' expr ')' | '-' factor"""
    tokens = tokenize(expr)
    pos = 0

    def peek():
        return tokens[pos] if pos < len(tokens) else None

    def take():
        nonlocal pos
        pos += 1
        return tokens[pos - 1]

    def parse_expr():
        value = parse_term()
        while peek() in ("+", "-"):
            op = take()
            rhs = parse_term()
            value = value + rhs if op == "+" else value - rhs
        return value

    def parse_term():
        value = parse_factor()
        while peek() in ("*", "/"):
            op = take()
            rhs = parse_factor()
            value = value * rhs if op == "*" else value / rhs
        return value

    def parse_factor():
        tok = take()
        if tok == "(":
            value = parse_expr()
            if take() != ")":
                raise ValueError("expected )")
            return value
        if tok == "-":                          # unary minus
            return -parse_factor()
        return float(tok)

    result = parse_expr()
    if pos != len(tokens):
        raise ValueError("unexpected token " + str(tokens[pos]))
    return result

def tokenize(s):
    out, i = [], 0
    while i < len(s):
        ch = s[i]
        if ch.isspace():
            i += 1
        elif ch.isdigit() or ch == ".":
            j = i
            while j < len(s) and (s[j].isdigit() or s[j] == "."):
                j += 1
            out.append(s[i:j]); i = j
        elif ch in "+-*/()":
            out.append(ch); i += 1
        else:
            raise ValueError("bad character " + ch)
    return out
```

```
# 1 + 2 * 3 = 7.0;  (1 + 2) * 3 = 9.0;  -(2 + 3) * -2 = 10.0;  7 - 2 - 1 = 4.0
# 3,000 random nested expressions agree with Python's eval
```

How the code works, step by step.

1. **tokenize** turns the string into numbers and single-character operators, skipping spaces.
2. **parse_expr** reads one term, then keeps reading "+ term" or "− term", applying each immediately. Applying left to right is what makes $7 - 2 - 1$ equal 4, not 6.
3. **parse_term** is the same one level down, with * and /.
4. **parse_factor** handles a number, a parenthesised sub-expression (recursing back to the top level) or a unary minus.
5. After parsing, any leftover token means the input was malformed.

Complexity. O(length of the expression): each token is read once.

How to test it live. Precedence ($1 + 2 * 3$), parentheses, left-to-right subtraction ($7 - 2 - 1$), unary minus, and comparison with eval on random expressions.

P24. Shortest path through a maze given as strings

Reported: *Glassdoor, Quantitative Trader coding challenge, Nov 2021 ("navigate through a 'maze' represented as an array of strings"); Glassdoor reports of BFS problems in HRT first rounds, 2023 and 2024*

> **ALSO IN OA PREP** Related to, but not the same as, **H3** (maze with portals) in your OA prep document — that one adds teleporting cells to the same BFS idea.

The task.

- The maze is a list of strings with S (start), E (exit), # (wall) and . (open).
- Return the fewest up/down/left/right moves from S to E, or −1 if E cannot be reached.

The approach. Breadth-first search: explore cells in order of distance using a queue. The first time E is taken from the queue, its distance is the shortest.

```python
from collections import deque

def shortest_path(maze):
    """maze: list of strings, 'S' start, 'E' exit, '#' wall, '.' open.
    Return the fewest moves (up/down/left/right) from S to E, or -1."""
    R, C = len(maze), len(maze[0])
    start = next((r, c) for r in range(R) for c in range(C) if maze[r][c] == "S")
    dist = {start: 0}
    queue = deque([start])
    while queue:
        r, c = queue.popleft()              # FIFO: cells come out in order of distance
        if maze[r][c] == "E":
            return dist[(r, c)]
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and maze[nr][nc] != "#" and (nr, nc) not in dist:
                dist[(nr, nc)] = dist[(r, c)] + 1   # mark when queued, not when popped
                queue.append((nr, nc))
    return -1
```

```python
# the 4 x 8 example -> 14;  ["S#E"] -> -1;  ["SE"] -> 1
```

How the code works, step by step.

1. Find S, give it distance 0 and put it in a **deque** (a queue with O(1) pops from the front).
2. Pop the oldest cell. Because cells enter in order of distance, the first time E is popped its distance is minimal.
3. For each of the four neighbours inside the grid and not a wall, if it has no distance yet, set it to current + 1 and enqueue it.
4. Mark distance **when enqueuing**, not when popping — otherwise the same cell can be queued many times.

Complexity. $O(R \times C)$: every cell is queued at most once.

How to test it live. S next to E; an unreachable E; a maze with two routes of different lengths.

P25. Count inversions in O(n log n)

Reported: *PracHub, HRT Data Scientist technical screen, Nov 2025 — paired with "derive the expected number of inversions" (question C5 in Part 3)*

> **ALSO IN OA PREP** Same problem as **E2** in your OA prep document. New here: the link to the expected-inversions derivation (C5) and the step-by-step explanation.

The task.

– Given an array, count pairs $i < j$ with $a[i] > a[j]$, faster than checking every pair.

The approach. Merge sort. While merging two sorted halves, when an element from the right half is placed before the remaining left-half elements, it forms an inversion with every one of them — count them all at once.

```python
def count_inversions(a):
    """Pairs i < j with a[i] > a[j], in O(n log n) by merge sort."""
    def sort_count(xs):
        if len(xs) <= 1:
            return xs, 0
        mid = len(xs) // 2
        left, inv_l = sort_count(xs[:mid])
        right, inv_r = sort_count(xs[mid:])
        merged, inv, i, j = [], inv_l + inv_r, 0, 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i]); i += 1
            else:
                merged.append(right[j]); j += 1
                inv += len(left) - i       # right[j] is smaller than every remaining left item
        merged += left[i:] + right[j:]
        return merged, inv
    return sort_count(list(a))[1]
```

```python
# matches the O(n^2) count on 2,000 random arrays; the average over all 720 permutations of 6 is 7.5 = 6*5/4
```

How the code works, step by step.

1. Split the array in half and recursively sort and count each half.
2. Merge with two pointers. If $left[i] \le right[j]$, take $left[i]$ — no new inversion.
3. Otherwise $right[j]$ is smaller than $left[i]$ **and** every element after it in the (sorted) left half: add **len(left) − i** inversions in one step, then take $right[j]$.
4. Using $\le$ (not $<$) keeps equal values from being counted as inversions.

Complexity. $O(n \log n)$ time, $O(n)$ extra memory.

How to test it live. Compare with the double loop on small random arrays; the sorted array gives 0 and the reversed array gives $n(n-1)/2$.

> **KEY** This ties the two halves of the interview together: compute $E[\text{inversions}] = n(n-1)/4$ by hand (C5), then check it with this code — the exact "derive, then verify" pattern.

P26. Find a local minimum in 1-D and 2-D

Reported: *PracHub, HRT Software Engineer technical screen, Jul 2026*

> **ALSO IN OA PREP** Same problem as **I2** in your OA prep document. This version adds a step-by-step explanation of both binary searches.

The task.

- 1-D: return an index whose value is ≤ its neighbours (ends compare with their single neighbour), faster than O(n).
- 2-D: return a cell ≤ its four neighbours.

The approach. Binary search on "downhill". If a[mid] > a[mid + 1], walking right goes downhill and must reach a local minimum before the end; otherwise one exists at mid or to its left. In 2-D, take the minimum of the middle column and move toward a smaller horizontal neighbour.

```python
def local_min_1d(a):
    """Index i with a[i] <= both neighbours (ends compare with one). O(log n).
    Invariant: a local minimum always exists inside [lo, hi] because we only
    move toward a smaller neighbour, i.e. 'downhill'."""
    lo, hi = 0, len(a) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] > a[mid + 1]:
            lo = mid + 1          # downhill to the right: a minimum lies right of mid
        else:
            hi = mid              # a[mid] <= a[mid+1]: a minimum lies at mid or to its left
    return lo

def local_min_2d(g):
    """Cell <= its 4 neighbours. Binary search on columns: take the column's
    minimum, then move toward a smaller horizontal neighbour. O(rows * log cols)."""
    R, C = len(g), len(g[0])
    lo, hi = 0, C - 1
    while True:
        mid = (lo + hi) // 2
        r = min(range(R), key=lambda i: g[i][mid])        # smallest in this column
        left = g[r][mid - 1] if mid > 0 else float("inf")
        right = g[r][mid + 1] if mid < C - 1 else float("inf")
        if g[r][mid] <= left and g[r][mid] <= right:
            return r, mid                                  # also <= up/down: it is the column min
        if left < g[r][mid]:
            hi = mid - 1
        else:
            lo = mid + 1
```

```python
# 3,000 random arrays and grids (with ties): every returned position is a valid local minimum
```

How the code works, step by step.

1. 1-D invariant: the range [lo, hi] always contains a local minimum, because we only ever move toward a smaller neighbour.
2. If **a[mid] > a[mid + 1]**, the right side slopes down from mid, so set lo = mid + 1. Otherwise set hi = mid. When lo == hi, that index is a local minimum.
3. 2-D: in the middle column, find the row with the smallest value. It is already ≤ its up and down neighbours.
4. If it is also ≤ its left and right neighbours, return it. Otherwise move to the half containing the smaller horizontal neighbour — the same downhill argument as 1-D, applied to columns.

Complexity. 1-D: O(log n). 2-D: O(R log C) — each step scans one column.

How to test it live. Strictly increasing, strictly decreasing, a single element, all equal values, and random arrays checked against the definition.

P27. Return the k smallest numbers

Reported: *PracHub, HRT Software Engineer technical screen, May 2026 ("compare full sorting, a size-k max-heap, quickselect"); Glassdoor, Algorithm Developer onsite, Nov 2015 (k-th largest)*

> **ALSO IN OA PREP** Same problem as **D4** in your OA prep document. This version adds the quickselect method alongside the heap, with a step-by-step explanation.

The task.

– Return the k smallest integers of an unsorted list, in ascending order, keeping duplicates. Discuss the trade-offs between approaches.

The approach. Three answers, in the order to present them: (1) sort and slice — O(n log n), simplest; (2) a max-heap of size k — O(n log k), streams and uses O(k) memory; (3) quickselect — O(n) on average.

```python
import heapq, random

def k_smallest_heap(nums, k):
    """Keep a max-heap of the k smallest seen so far: O(n log k) time, O(k) memory."""
    if k <= 0:
        return []
    heap = []                                   # negated values -> max-heap
    for x in nums:
        if len(heap) < k:
            heapq.heappush(heap, -x)
        elif x < -heap[0]:                      # smaller than the largest kept value
            heapq.heapreplace(heap, -x)
    return sorted(-v for v in heap)

def k_smallest_select(nums, k):
    """Quickselect: average O(n). Partition around a random pivot, recurse on one side."""
    if k <= 0:
        return []
    a = list(nums)
    def select(lo, hi, k):                      # put the k smallest of a[lo:hi] first
        if hi - lo <= 1:
            return
        pivot = a[random.randint(lo, hi - 1)]
        less = [x for x in a[lo:hi] if x < pivot]
        equal = [x for x in a[lo:hi] if x == pivot]
        more = [x for x in a[lo:hi] if x > pivot]
        a[lo:hi] = less + equal + more
        if k < len(less):
            select(lo, lo + len(less), k)
        elif k > len(less) + len(equal):
            select(lo + len(less) + len(equal), hi, k - len(less) - len(equal))
    select(0, len(a), k)
    return sorted(a[:k])
```

```python
# k_smallest_heap([5, 1, 4, 1, 3, 9], 3) -> [1, 1, 3]; both methods agree with sorted(a)[:k] on 3,000 random cases
```

How the code works, step by step.

1. **Heap**: keep the k smallest values seen so far in a max-heap (negated values). The top is the largest of them.
2. A new value smaller than the top replaces it (**heapreplace** pops and pushes in one step); larger values are ignored.
3. **Quickselect**: split around a random pivot into less / equal / more. If the k smallest all lie in "less", recurse there; if they include all of "less" and "equal", recurse into "more" for the rest; otherwise the split already puts the k smallest first.
4. Grouping values **equal** to the pivot is what keeps duplicates from causing infinite recursion.

Complexity. Heap: $O(n \log k)$ time, $O(k)$ memory. Quickselect: $O(n)$ on average, $O(n^2)$ worst case (very unlikely with a random pivot).

How to test it live. $k = 0$, $k = n$, many duplicates, negative numbers; compare with $\text{sorted}(a)[:k]$.

Part 5 — Python cheat sheet: syntax to memorize

Everything below is used somewhere in Parts 3–4 or in your OA prep. Verified on Python 3.10+. The point is not to learn new features — it is to never spend thirty seconds recalling which end randint includes or which direction bisect returns. Random and numpy come first because the programming exercise is most likely a simulation of the probability question.

Imports — type these first, before reading the problem

```
import random
import numpy as np
from collections import defaultdict, deque, Counter
import heapq, bisect, itertools, math
```

random — simulation, one trial at a time

```
random.seed(0)               # reproducible: same numbers on every run
random.random()              # float in [0, 1)
random.randint(1, 6)         # integer 1..6, BOTH ends included -- a die
random.randrange(6)          # integer 0..5, end EXCLUDED (like range)

random.choice(['H', 'T'])                    # one item, equally likely
random.choices('HT', weights=[0.6, 0.4], k=10)  # k draws WITH replacement, biased
random.sample(range(52), 5)   # 5 distinct items, WITHOUT replacement (deal a hand)
random.shuffle(deck)          # shuffles the list IN PLACE, returns None

random.uniform(2, 5)          # float in [2, 5]
random.gauss(0, 1)            # normal: mean 0, sd 1
random.expovariate(2.0)       # exponential, rate 2 (mean 1/2)
random.random() < p           # an event with probability p; True sums as 1
```

TRAP randint(1, 6) includes 6, but randrange(6) and range(6) stop at 5. And deck = random.shuffle(deck) sets deck to None — shuffle works in place.

numpy — simulation, many trials at once

```
rng = np.random.default_rng(0)               # modern generator, seeded
d = rng.integers(1, 7, size=(100_000, 2))    # 100,000 rolls of two dice; HIGH END EXCLUDED
np.mean(d[:, 0] == d[:, 1])                  # P(match)   -> about 0.166 (exact 1/6)
np.mean(d.sum(axis=1) == 7)                  # P(sum = 7) -> about 0.167

rng.random(5); rng.normal(0, 1, 5)           # uniforms; normals
rng.choice([1, 2, 3], size=4)                # sampling with replacement
rng.geometric(1/6, size=200_000).mean()      # rolls until first success -> about 6.0

x.mean(axis=1); np.median(x, axis=1)         # one value per row (per data set)
np.var(x); np.var(x, ddof=1)                 # divide by n; divide by n - 1
np.cumsum([1, 2, 3])                         # [1 3 6] running totals
```

TRAP rng.integers(1, 7) gives 1–6 — the high end is excluded, the opposite of random.randint(1, 6). A comparison such as d[:, 0] == d[:, 1] gives an array of True/False; its mean is the estimated probability. Use plain random for "roll until something happens" loops (each game has a different length); use numpy when every trial has the same shape.

Dictionaries and counting

```
d = defaultdict(int)          # d[k] += 1 with no KeyError
g = defaultdict(list)         # g[k].append(x) -- the grouping idiom
```

```
d.get(k, 0)                  # read with a default, no insertion

c = Counter(seq)             # counts in one pass
c.most_common(2)             # [(value, count), ...] highest first
Counter('aab') - Counter('ab')            # Counter({'a': 1}); negatives dropped
sum(v - 1 for v in c.values() if v > 1)    # "how many duplicates" in one line
```

Heaps

```
heapq.heappush(h, x); heapq.heappop(h)     # MIN-heap only
heapq.heapify(lst)                        # O(n), in place
heapq.heappushpop(h, x)     # push then pop  -- cheaper than two calls
heapq.heapreplace(h, x)     # pop then push  -- NOT the same result
heapq.nsmallest(k, xs); heapq.nlargest(k, xs)
heapq.heappush(h, -x)       # max-heap: negate going in and coming out
heapq.heappush(h, (price, time, qty))     # tuples compare left to right
```

TRAP heappushpop and heapreplace are not interchangeable. On heap [1,5,3]: heappushpop(h, 0) returns 0 and leaves the heap unchanged; heapreplace(h, 0) returns 1 and inserts 0. Use heappushpop when the new value might be smaller than the root, heapreplace when the root must always be evicted.

Binary search on a sorted list

```
bisect.bisect_left(a, v)      # first index i with a[i] >= v
bisect.bisect_right(a, v)     # first index i with a[i] >  v
bisect.insort(a, v)           # insert, keeping sorted (O(n) shift)

# on a = [1,2,2,2,3]:  bisect_left(a,2) == 1,  bisect_right(a,2) == 4
# count of v in a  ==  bisect_right(a,v) - bisect_left(a,v)
```

TRAP Picking the wrong one is the most common bisect error. Say it as "left finds where v starts, right finds where v ends." For "is there any element in [lo, hi]?" use bisect_left(a, lo) then test a[i] <= hi — that is exactly OA prep problem B2.

Deques and queues

```
dq = deque()
dq.append(x); dq.appendleft(x)
dq.pop();     dq.popleft()          # both O(1)
dq = deque(maxlen=3)                # auto-evicts from the left when full
```

TRAP list.pop(0) is O(n) because every remaining element shifts. Using a list as a queue turns an O(n) algorithm into $O(n^2)$ and is a silent cause of timeouts. Always deque.

Sorting

```
xs.sort()                           # in place, returns None
sorted(xs)                          # returns a new list
sorted(xs, key=lambda t: t[1])                # by second field
sorted(xs, key=lambda t: (t[0], -t[1]))       # asc, then desc: explicit tie-break
sorted(xs, reverse=True)
# Python's sort is STABLE: equal keys keep their original order
```

TRAP For a sweep line (P20 here, B1 in your OA prep), do not rely on −v sorting ahead of +v to break ties — it works for half-open intervals by accident. Write the key explicitly so the intent survives a re-read.

Grids

```
grid = [[0] * C for _ in range(R)]        # CORRECT
grid = [[0] * C] * R                      # WRONG: R references to ONE row

DIRS = ((0,1), (0,-1), (1,0), (-1,0))           # orthogonal
DIAG = ((1,1), (1,-1), (-1,1), (-1,-1))         # diagonal only
if 0 <= nr < R and 0 <= nc < C: ...             # bounds check

copy = [row[:] for row in grid]           # deep enough for a 2-D int grid
```

TRAP [[0]*C]*R is the most destructive one-liner in grid problems. grid[0][0] = 9 sets column 0 of every row, because all R rows are the same list object. It stays invisible until your output is mysteriously symmetric — directly relevant to grid simulations such as P18 and P21.

Arithmetic

```
-7 // 2        # -4  (floors toward minus infinity)
int(-7 / 2)    # -3  (truncates toward zero)
divmod(-7, 2)  # (-4, 1)

float('inf'), float('-inf'), math.inf       # sentinel initial values
INF = float('inf')                          # "no answer yet"
```

TRAP Python's // floors rather than truncating, so it disagrees with C++ and Java on negative operands. If a problem has negative coordinates and midpoints — an infinite number line, say — this changes answers.

Iteration helpers

```
for i, x in enumerate(xs): ...
for i, x in enumerate(xs, start=1): ...
for a, b in zip(xs, ys): ...
for x in reversed(xs): ...

itertools.accumulate([1,2,3,4])      # 1, 3, 6, 10   -- running prefix sums
itertools.pairwise([1,2,3])          # (1,2), (2,3)  -- adjacent pairs, 3.10+
itertools.product(range(1, 7), repeat=2)   # all 36 dice outcomes -- exact checks

max(xs, default=0)                   # no ValueError on an empty sequence
min(xs, key=lambda t: t[1], default=None)
any(...), all(...)
```

TRAP max() and min() raise ValueError on an empty sequence. default= is shorter and safer than an if-guard, and empty input is a standard hidden test case.

Strings

```
s[::-1]                    # reverse
s[:k][::-1] + s[k:]        # reverse the first k characters (OA prep A3)
''.join(parts)             # build strings this way
''.join(sorted(s))         # canonical anagram key

s.isdigit(); s.isalpha(); s.lower()
int(str(x)[::-1])          # digit reversal (OA prep E1)
```

TRAP Building a string with s += ch in a loop is $O(n^2)$: strings are immutable, so each += copies the whole thing. Append to a list and call join once.

Deep recursion

```python
import sys
sys.setrecursionlimit(10**6)     # default is ~1000; deep recursion (e.g. a long tree, P17) dies silently
```

Part 6 — Sources, and how much weight each deserves

Everything in Parts 3 and 4 is candidate-reported, not official. Glassdoor entries were read in full (530 HRT interview reviews, 2015–Oct 2026) and are cited by role and month. Prep sites (QuantVault, Quantt, Tradermath, LinkJob, DEV Community) rewrite candidate reports and add their own examples, so their questions are less certain; where a prep site wrote its own example rather than reporting one, it is not used as evidence here.

Source	What it contributed	Weight
Wall Street Oasis WiTTI thread	The only WiTTI-specific report: the invite "says Probability and a programming exercise." No questions posted.	Format only
Glassdoor — HRT interviews (all roles)	Most dated questions in Parts 3–4; the closest one-hour intern and sophomore analogues in Part 1.	Highest: first-hand, dated
Glassdoor — WiTTI page	One review (Nov 2023): OA was LeetCode easy/medium, "significantly easier than HRT's normal interview process."	First-hand, single data point
HRT — interview guide (HRT Beat blog)	What HRT says it evaluates; when to over-communicate.	Official, about engineers
Quantt HRT internship and interview guides	Jan 2026 "stats + simulation" report; summer intern final-round structure (Part 1); fancy numbers (P15); two-sixes-in-a-row example (A6); 2027 WiTTI dates.	Medium: aggregator
Tradermath HRT guide	"Derive a strategy, then validate it by simulation" as a common format; Romeo-and-Juliet example.	Medium: aggregator
QuantVault HRT question list and process page	Dated topic list confirming many Part 3 classics; "estimator then simulate" pattern.	Medium: rewritten prompts
PracHub — HRT questions and interview experiences	Rewritten statements plus linked first-hand reports for P16–P22 and P25–P27 (2025–2026 technical screens, including a Jul 2026 intern round).	Medium: rewritten, but tied to dated first-hand reports
Reddit r/quantfinance, r/learnquant (quantprof.org images)	Sum of 10 dice divisible by 7 (A7), 10 chords (C6), last two uniforms (C7), single noodle loop (F5).	Lower: prep-site posts labelled "HRT"
DEV Community HRT video-interview write-up	Kadane question and the sum-of-7 follow-ups (A2, P7).	Lower: coaching-service post
LinkJob HRT quant write-up	Gambler's ruin wording, broken stick, n+1 coins, interval flips, sum > product.	Lower: marketing post

Links

- Wall Street Oasis — "Interview – HRT WITTI – Women in Technology Trading Winternship"
  `https://www.wallstreetoasis.com/forum/trading/interview-hrt-witti-women-in-technology-trading-winternship`
- Glassdoor — Hudson River Trading interview questions (all roles)
  `https://www.glassdoor.com/Interview/Hudson-River-Trading-Interview-Questions-E470937.htm`
- Glassdoor — HRT Women in Trading Internship interviews
  `https://www.glassdoor.com/Interview/Hudson-River-Trading-Women-in-Trading-Internship-Interview-Questions-EI_IE470937.0,20_KO21,48.htm`
- Glassdoor — HRT Algorithm Developer Intern interviews
  `https://www.glassdoor.com/Interview/Hudson-River-Trading-Algorithm-Developer-Intern-Interview-Questions-EI_IE470937.0,20_KO21,47.htm`
- HRT — How to Prepare for Your Software Engineer Interview at HRT
  `https://www.hudsonrivertrading.com/hrtbeat/interview-at-hrt/`

- HRT — WiTTI Winter 2026 posting (The Muse mirror)
  https://www.themuse.com/jobs/hudsonrivertrading/women-in-trading-and-technology-internship-witti-winter-2026
- Quantt — HRT Internship 2027: Pay, Dates, OA and Interviews
  https://www.quantt.co.uk/resources/hrt-internship
- Quantt — HRT Interview: Process, Algo Developer & SWE Questions
  https://www.quantt.co.uk/resources/hudson-river-trading-interview
- Tradermath — Hudson River Trading Interview Guide
  https://www.tradermath.org/articles/hudson-river-trading-interview-guide
- QuantVault — Hudson River Trading interview questions (106 problems)
  https://quantvault.org/hrt-interview-questions.html
- QuantVault — Hudson River Trading interview process
  https://quantvault.org/hrt-interview-process.html
- SpaceComplexity — HRT onsite interview, every round explained
  https://spacecomplexity.ai/blog/hudson-river-trading-onsite-interview
- Reddit r/learnquant — "HRT quant interview question" (1)
  https://www.reddit.com/r/learnquant/comments/1wiyybs/hrt_quant_interview_question/
- Reddit r/quantfinance — "HRT quant interview question" (2)
  https://www.reddit.com/r/quantfinance/comments/1v728ln/hrt_quant_interview_question/
- Reddit r/quantfinance — "HRT quant interview question" (3)
  https://www.reddit.com/r/quantfinance/comments/1v0wmga/hrt_quant_interview_question/
- Reddit r/learnquant — "HRT quant interview question" (4)
  https://www.reddit.com/r/learnquant/comments/1wwyczv/hrt_quant_interview_question/
- PracHub — Hudson River Trading question list
  https://prachub.com/companies/hudson-river-trading
- PracHub — HRT intern SWE interview: tree path-sum question (Jul 2026)
  https://prachub.com/interview-experiences/hudson-intern-software-engineer-interview-experience-a-leetcode-easy-tree-path-question-with-no-cplusplus-trivia
- PracHub — HRT Data Scientist interview: mean versus median estimation (Sep 2026)
  https://prachub.com/interview-experiences/hrt-data-scientist-interview-experience-an-oa-and-a-mean-versus-median-estimation-interview
- PracHub — HRT SWE interview: 30 minutes, misread one rule (Aug 2026)
  https://prachub.com/interview-experiences/hudson-river-trading-software-engineer-interview-experience-30-minutes-misread-one-rule-no-time-to-fix
- PracHub — Ball position in a tilting maze
  https://prachub.com/coding-questions/ball-position-in-a-tilting-maze-after-timed-tilt-events
- PracHub — Watcher simulation and integer conversion
  https://prachub.com/coding-questions/solve-watcher-simulation-and-integer-conversion
- PracHub — Street lamps
  https://prachub.com/coding-questions/smallest-integer-point-lit-by-the-most-street-lamps-on-a-line
- PracHub — Maximum diamond sum
  https://prachub.com/coding-questions/maximum-sum-of-a-filled-diamond-that-fits-inside-an-integer-grid
- PracHub — Wordle solver
  https://prachub.com/coding-questions/implement-wordle-style-word-guessing-solver
- DEV Community — Hudson River Trading VO interview experience
  https://dev.to/net_programhelp_e160eef28/hudson-river-trading-vo-interview-experience-gch
- LinkJob — HRT quant interview experience and prep tips
  https://www.linkjob.ai/interview-questions/hrt-quant-interview/

KEY Your OA preparation (HRT WiTTI CodeSignal OA Prep) is still useful here: its sections on grid simulation, heaps and hash counting cover the kind of short implementation a coding exercise may ask for. Six problems appear in both documents — P9 (J3), P17 (F1), P19 (H2), P25 (E2), P26 (I2) and P27 (D4) — and are marked "Also in OA prep" where they appear. They are kept here so this document can be studied on its own; the versions here add follow-ups and step-by-step explanations. P24 is related to H3 but different.